

280 mondat — záróvizsga beszédvázlat

EKKE Programtervező Informatikus BSc · Záróvizsga · 14 tétel × 2 alpont × 10 mondat

280 mondat — záróvizsga beszédvázlat

Alpontonként 10 összefüggő, kimondható mondat. A hivatalos kérdés minden topicját érinti, sorrendben olvasva folyamatos beszéd.

1.a Bevezetés az informatikába

1. Az **információ** új ismereteket hordozó jelek tartalmi befogadása, ami cselekvésre késztet; az **adat** ennek a hordozója, önmagában jelentés nélküli.
2. Az információ útja: **adó** → **kódolás** → **csatorna (zaj)** → **dekódolás** → **vevő**, ezért a kódolás-dekódolás minőség kulcsfontosságú.
3. Az információ mértékegysége a **bit**: két egyenlően valószínű jel egyikének kiválasztásához tartozó információmennyiség.
4. Az **entrópia** a rendszer bizonytalanságát méri, képlete $H = - \sum p_i \cdot \log_2 p_i$, ahol a valószínűségek összege 1.
5. Az **átlagos kódhossz** $\sum p_i \cdot h_i$, a **kódolási hatások** ennek és az entrópiának a hányadosa — jó kódolásnál közel 1.
6. A **Shannon-Fano** eljárás gyakoriság szerint csökkenően rendez, közel egyenlő részekre osztja a listát és 0/1-et rendel, rekurzívan halad.
7. A **Huffman-kódolás** összevonja a két legkisebb valószínűségű jelet és visszafelé olvassa ki a kódokat — optimális prefix kódot ad.
8. **Fixpontos számábrázolás**: bináris, a legmagasabb bit az előjel, a negatív számokat kettes komplementssel ábrázoljuk.
9. **Lebegőpontos (IEEE 754, 32 bit)**: 1 előjelbit + 8 karakterisztika (eltolásos +127) + 23 mantissza (implicit bittel).
10. A **karakterkódolás** ASCII (128 karakter) vagy Unicode (UTF-8) — a gép számára teszi értelmezhetővé a szöveget.

Történet

Az informatika legalapvetőbb fogalma az **információ** — új ismereteket hordozó jelek befogadása, amelyek cselekvésre vagy döntésre késztetnek minket. Az információt önmagában nem érzékeljük; az **adatok** hordozzák, és ezek önmagukban jelentés nélküliek — az értelmezés tölti meg őket tartalommal. Bármilyen információ útja egy szabályos lánc szerint zajlik: az adó kódolja az üzenetet, a csatornán keresztül átküldi (közben zaj torzíthatja), majd a vevő dekódolja és értelmezi. Hogy mindezt számszerűsíteni tudjuk, bevezettük a **bit** fogalmát: két egyformán valószínű választás egyikét jelenti, és ez az információ alapegysége. Egy információs forrás bizonytalanságát az **entrópia** képletével írjuk le: $H = - \sum p_i \log_2 p_i$, ahol p_i az egyes jelek kibocsátási valószínűsége. Az átlagos kódhossz és a **kódolási hatások** azt mutatja, mennyire jól használjuk ki a kódot — a hatások 1-hez közele a jó kódolás esetén. A gyakorlatban két klasszikus kódolási eljárás alakult ki: a **Shannon-Fano** módszer a gyakoriság szerint rendezett jeleket osztja két közel egyenlő valószínűségű részre, és 0/1 prefixet rendel rekurzívan; a **Huffman** pedig fát épít alulról, mindig a két legkisebb valószínűségű jelet vonva össze, és így optimális prefix-kódot eredményez. Számokat a gép vagy **fixpontosan** tárol (binárisan, a legmagasabb bit az előjel, negatív számok kettes komplementsen), vagy **lebegőpontosan**, az IEEE 754 szabvány szerint: 32 biten egy előjelbit, nyolc karakterisztika (eltolásos +127), huszonhárom

mantissza-bit, az implicit bit elhagyásával. A karaktereket szintén számokká kódoljuk: az **ASCII** az alap 128 karaktert fed le (0–127), a **Unicode** (UTF-8) pedig gyakorlatilag minden írásrendszert.

1.b Elsőrendű logika

1. Az elsőrendű logika **paraméteres állításokkal (predikátumokkal)** dolgozik, amik termeken értelmezettek — termék: konstansok, változók, függvények.
2. Két **kvantor** van: $\forall$ ("minden x-re teljesül") és $\exists$ ("van olyan x, amelyre teljesül"). A logikai nyelv egy $\langle \text{Var}, \text{Pr}, \text{Fn} \rangle$ formális hármas: változók, predikátumok (nem üres) és függvények halmaza, mindegyik P-hez és F-hez rögzített paraméterszámmal.
3. A **szintaxis** a helyes formulák nyelvtana, a **szemantika** az értelmezés: az interpretáció definiálja a **domaint** és minden szimbólumhoz értéket rendel.
4. Egy formula **tautológia** (minden interpretációban igaz), **kontradikció** (mindig hamis) vagy **kielégíthető** (legalább egyben igaz).
5. A **KNF-re hozás 6 lépése**: implikáció eltüntetése, változótiszta alak, prenexizálás, Skolemizálás, KNF-re hozás disztributivitással, klózkra bontás.
6. **Skolemizálás**: az $\exists x$ kvantort eltüntetjük, és x helyébe új **Skolem-függvényt** írunk, amelynek paraméterei az előtte álló $\forall$ kvantorok változói; $\forall$ nélkül Skolem-konstans.
7. A **rezolúció** cáfoló eljárás: a bizonyítandó állítás tagadásából indulunk, és levezetjük az üres klózt (UNSAT).
8. Az **unifikáció** karakterről karakterre egyformává tesz két literált — változót konstanssal/függvénnyel illeszt, az **occur check** szerint $x \rightarrow f(x)$ tilos.
9. A **lineáris rezolúció** mindig az előző rezolvensre épít, és teljes algoritmus. Az **SLD rezolúció** szigorúbb (a melléklóznak az input halmazból kell jönnie), és csak **Horn-klózkra** teljes — ez olyan klóz, amelyikben legfeljebb 1 pozitív literál van, és a Prolog ezzel dolgozik.
10. A **Prolog** az első logikai programozási nyelv: Horn-klózokkal, SLD-rezolúcióval dönti el, hogy egy célformula logikai következménye-e a tényeknek és szabályoknak.

Történet

Az elsőrendű logika lehetővé teszi, hogy ne csak egyszerű igaz/hamis állításokkal dolgozzunk, hanem objektumok tulajdonságaival és a köztük lévő kapcsolatokkal is. Ehhez bevezeti a **predikátumokat**, amelyek paraméteres állítások — például a Szeret(x, y) azt mondja, hogy x szereti y-t. A paraméterek **termek**, és háromfélék: konstansok (konkrét objektumok), változók (általánosítást tesznek lehetővé), és függvények (egy termből másik termet adnak vissza, pl. anya(x)). Két **kvantorral** fejezünk ki általánosságot: a $\forall$ ("minden x-re teljesül") és a $\exists$ ("van olyan x, amelyre teljesül"). A logikai nyelv formálisan egy $\langle \text{Var}, \text{Pr}, \text{Fn} \rangle$ hármas: változók, predikátumok (nem üres), függvények — mindegyikhez tartozik egy paraméterszám. A szemantikához **interpretációt** rendelünk: meghatározzuk az objektumtartományt (domain), és minden konstansnak, predikátumnak, függvénynek konkrét értéket adunk. Egy formula lehet **tautológia** (minden interpretációban igaz), **kontradikció** (mindig hamis), vagy **kielégíthető** (legalább egyben igaz). A logikai következménynél egy állítás akkor következik a premisszákból, ha azokkal együtt a tagadott következménnyel az **üres klózt** levezethetjük. Ez a **rezolúció**: cáfoló eljárás, ami a bizonyítandó állítás tagadásából indul, és ha üres klózt vezet le, akkor az állítás igaz. Mielőtt rezolválunk, **KNF-re kell hozni** a formulát hat lépésben: implikációk eltüntetése

$(A \rightarrow B \equiv \neg A \vee B)$, változótiszta alakra hozás, prenexizálás (kvantorok előrehozása), **Skolemizálás** ($\exists$ kvantorok helyettesítése új Skolem-függvénnyel, vagy ha nincs előtte $\forall$, konstanssal), KNF-re hozás disztributivitással, klózokra bontás. A rezolváláshoz **unifikációra** is szükségünk lehet, ami változók helyettesítésével karakterre egyformává teszi a literálokat — az "occur check" tiltja, hogy egy változót olyan termre cseréljünk, amiben ő szerepel ($x \rightarrow f(x)$ tilos). A **lineáris rezolúció** mindig az előző rezolvensre épít, és bármilyen UNSAT klózalmazból levezeti az üres klózt — teljes algoritmus. Az **SLD rezolúció** szigorúbb: a melléklóznak mindig az input klózalmazból kell jönnie; nem minden klózalmazra működik, de **Horn-klózok esetén teljes** — ahol Horn-klóz az, amelyikben legfeljebb egy pozitív literál van. Pontosan ezt használja a **Prolog**, a legelső logikai programozási nyelv: tényekből és szabályokból álló programja eldönti, hogy egy célformula logikai következménye-e az inputnak.

2.a Magasszintű programozási nyelvek I

1. A **tömb és lista** homogén, véletlen elérésű, folytonos adatszerkezet — a tömb fix méretű, a `List<T>` dinamikusan bővül.
2. A **rekord** heterogén mezőket tartalmaz; C#-ban a **class** referenciatípus (heap, GC), a **struct** értéktípus (stack, érték szerint másolódik).
3. A **felsorolásos típus (enum)** megnevezett konstansok halmaza, alapból int-re fordul.
4. A **metódus** szignatúrával rendelkezik (név + paramétertípusok), lehet **static** (osztálysztű) vagy példányszintű, és **túlterhelhető**.
5. **Paraméterátadás módjai**: érték szerint (alap eset), **ref** (referencia, be+ki), **out** (kötelező kimenet), **in** (csak olvasható), **params** (változó paraméterszám tömbként).
6. Nyelvek **fordítási megoldásai**: **AOT** (Ahead-of-Time — előre gépi kódra fordít, pl. C, Rust), **értelmezett** (futás közben sor-sorra értelmez, pl. Bash), vagy **JIT** (Just-in-Time — futás közben fordít, pl. Java JVM, C# CLR).
7. A **.NET** fő összetevői: a **CLR** (Common Language Runtime) futtatja a programot — a **JIT** (Just-in-Time fordító) futás közben gépi kódra alakít, a **GC** (Garbage Collector) automatikusan szabadítja fel a memóriát. A **BCL** (Base Class Library) az alaposztálykönyvtár (List, Stream, stb.), az **IL** (Intermediate Language) pedig a platformfüggetlen közbenső kód, amire minden .NET nyelv (C#, F#, VB) fordul.
8. C# forrásból **IL** (közbenső kód) lesz, amit a **CLR** futás közben gépi kódra fordít, és kezeli a memóriát, típusbiztonságot, kivételeket.
9. A **JVM** (Java Virtual Machine) hasonló koncepció: Java és más JVM-re fordító nyelvek (Scala, Kotlin) **bytecode**-ra fordulnak, amit a JVM **JIT**-tel gépi kódra alakít — ugyanaz a séma, mint a .NET-ben az IL + CLR. A nyelvek **fordítási spektruma**: egyik végén az **AOT**-fordított nyelvek (Go, Rust — előre gépi kódra), másik végén a **tisztán értelmezett** nyelvek (CPython a standard Python implementáció), és köztük a **JIT**-esek (Java, C#).
10. A virtuális gépes nyelvek **platformfüggetlenséget** adnak — ugyanaz a kód több OS-en fut ugyanazzal a runtime-mal.

Történet

A magasszintű programozási nyelvek alapfogalmaival kezdjük. A **tömb és lista** összetett, homogén, véletlen elérésű, folytonos memóriájú adatszerkezet — a tömb fix méretű, a `List<T>` dinamikusan bővül. A **rekord** ezzel szemben heterogén mezőket tartalmaz, és C#-ban két formája van: a **class** referenciatípus, ami a heap-en él és a Garbage Collector kezeli; a **struct** értéktípus, ami a stack-en él és érték szerint másolódik. A **felsorolásos típus (enum)** megnevezett konstansok halmaza, alapból int-re fordul. A metódusok szignatúrával hívódnak, lehetnek **static** (osztálysztű) vagy példányszintűek, és **túlterhelhetők** ugyanazon a néven, de eltérő paramétertípusokkal. A paraméterek átadása többféleképpen történhet: alapértelmezetten érték szerint, **ref** módosítóval referencia szerint (be- és kimenetként), **out** módosítóval kötelező kimenetként, **in** módosítóval csak olvasható referenciaként, vagy **params** módosítóval változó számú argumentumot fogadva

tömbként. A nyelvek fordítási megoldásai három kategóriába esnek: az **AOT (Ahead-of-Time)** előre, a forrás futtatása előtt fordít gépi kódra (ilyen a C, Rust, Go); az **értelmezett** nyelvek futás közben soronként értelmezik a kódot (pl. Bash); a **JIT (Just-in-Time)** nyelvek pedig közbenső kódra fordítanak, amit a futtatókörnyezet futás közben fordít gépi kódra (ilyen a Java JVM-en és a C# .NET-en). A **.NET keretrendszer** fő összetevői: a **CLR (Common Language Runtime)** a futtatókörnyezet, ami JIT-tel fordítja az IL-t gépi kódra, **GC**-vel kezeli a memóriát, és biztosítja a típusbiztonságot és kivételkezelést. A **BCL (Base Class Library)** az alapsztálykönyvtár, ami a kollekciókat, IO-t, hálózati és szöveges osztályokat tartalmazza. Az **IL (Intermediate Language)** a közbenső, platformfüggetlen kód, amire minden .NET nyelv (C#, F#, VB.NET) fordul a saját compilerével. Ugyanez a séma működik a Java világban a JVM-en bytecode-dal — más nyelvek (Scala, Kotlin, Groovy) is JVM-re fordulnak.

2.b Adatszerkezetek és algoritmusok — programozási tételek

1. Az **algoritmus** véges, egyértelmű, általános, hatékony lépéssorozat a feladat megoldására.
2. **Megadási módok**: szöveges, pszeudokód, folyamatábra, struktogram (Nassi-Shneiderman), Jackson-diagram.
3. A **strukturált algoritmus** három alapszerkezete: **szekvencia**, **szelekció**, **iteráció** — goto nélkül.
4. **Sorozathoz elemi értéket** rendelő tételek: eldöntés, kiválasztás, lineáris keresés, megszámlálás, maximumkeresés, összegzés.
5. **Sorozathoz sorozatot** rendelők: másolás (transzformáció), kiválogatás, szétválogatás; **több sorozathoz egyet**: unió, metszet, összefuttatás.
6. **Rendezések**: buborék/beszúrásos/kiválasztásos $O(n^2)$; **quicksort/mergesort/heapsort** $O(n \log n)$ — a quicksort legrosszabb esete $O(n^2)$.
7. A **mergesort** stabil és mindig $O(n \log n)$, de $O(n)$ extra memória kell; a quicksort átlagosan gyors, in-place, de instabil.
8. A **visszalépéses keresés (backtracking)** rekurzív próbálkozás: zsákutcánál visszalépés és másik opció — N királynő, sudoku, gráfszínezés.
9. A **programozási tételek** átültethetők különböző homogén adatszerkezetekre: tömbön, láncolt listán, halmazon, fán értelmezhetők.
10. A **halmaz adatszerkezet** konstrukciói: rendezetlen lista ($O(n)$), rendezett tömb bináris kereséssel ($O(\log n)$), karakterisztikus függvény bit-vektorral ($O(1)$), hash-tábla, fa.

Történet

Az adatszerkezetek és algoritmusok témakör az algoritmus fogalmával kezdődik. Az **algoritmus** véges sok, egyértelmű, jól definiált lépésből álló eljárás, ami egy feladatot véges idő alatt megold; tulajdonságai: véges, determinisztikus, általános és hatékony. Megadhatjuk szöveges leírással, **pszeudokóddal**, folyamatábrával, struktogrammal (Nassi-Shneiderman) vagy Jackson-diagrammal. A strukturált algoritmus három alapszerkezetre épül: **szekvenciára** (lépések egymás után), **szelekcióra** (if/switch, feltétel alapján), és **iterációra** (while, for, do-while ciklusok) — goto nélkül, jól strukturáltan. A klasszikus **programozási tételek** három családot alkotnak. Az első család egy sorozathoz elemi értéket rendel: idetartozik az eldöntés (van-e ilyen elem), a kiválasztás (mi az első ilyen indexe), a lineáris keresés, a megszámlálás, a maximumkeresés és az összegzés. A második család sorozathoz sorozatot rendel: másolás transzformációval, kiválogatás (feltételnek megfelelő elemek új sorozatba), szétválogatás (két sorozatba). A harmadik család több sorozatból állít elő egyet: unió, metszet, és összefuttatás rendezett sorozatok esetén. A **rendezések** különböző hatékonyságúak: a buborék-, beszúrásos- és kiválasztásos rendezés $O(n^2)$ műveletet igényel; a **quicksort, mergesort és heapsort** átlagban $O(n \log n)$ -re jut. A mergesort stabil és mindig $O(n \log n)$, de $O(n)$ extra memóriát igényel; a quicksort átlagban gyors, in-place, de legrosszabb esetben $O(n^2)$ -re romolhat. A **visszalépéses keresés (backtracking)** rekurzív próbálkozás: ha egy út zsákutca, visszalépünk és más opciót próbálunk — ilyen az N királynő probléma vagy a sudoku. A **halmaz** adatszerkezetnek többféle

konstrukciója van: rendezetlen lista ($O(n)$ keresés), rendezett tömb bináris kereséssel ($O(\log n)$), karakterisztikus függvény bit-vektorral ($O(1)$, de csak véges univerzumra), hash-tábla, vagy kiegyensúlyozott bináris keresőfa.

3.a Adatbázisrendszerek I — modellek

1. Három klasszikus adatbázis-modell van: **hierarchikus** (fa, 1 szülő), **hálós** (több szülő, M:N), és a domináns **relációs modell**.
2. A relációs modell **táblákban** tárol adatot, a kapcsolatokat **kulcsok** és **idegen kulcsok** reprezentálják, halmazalapú SQL-lel kérdezhető.
3. **Kulcsfajták**: a **szuperkulcs** olyan attribútum-halmaz, amittől minden más attribútum egyértelműen meghatározott (sok lehet egy táblában); a **candidate** (kulcsjelölt) egy minimális szuperkulcs — ha bármit elveszel belőle, már nem szuperkulcs; a **primary key** (PK, elsődleges kulcs) az egyik kiválasztott candidate, egyedi és NOT NULL, táblánként csak egy; a **foreign key** (FK, idegen kulcs) másik tábla PK-jára mutat, ez a kapcsolatok kifejezésének eszköze.
4. **Kapcsolat-típusok**: 1:1 (személy ↔ útlevelel), 1:N (osztály ↔ tanuló), M:N (hallgató ↔ tantárgy) — utóbbi **kapcsolótáblával** valósul meg.
5. **Anomáliák** normalizálatlan táblákban: beszúrási (új adat csak másikkal vihető be), módosítási (több helyen kell javítani), törlési (más adat törlése értékes infót töröl).
6. A **funkcionális függőség** $X \rightarrow Y$: ha X megegyezik két sorban, Y is megegyezik; a **transzitivitás** miatt $X \rightarrow Y$ és $Y \rightarrow Z$ -ből $X \rightarrow Z$.
7. **1NF**: minden cella **atomi** (nem összetett, nem ismétlődő).
8. **2NF**: 1NF + minden nem-kulcs attribútum teljesen függ az **összes** elsődleges kulcstól (összetett kulcsnál releváns).
9. A **3NF** (harmadik normálforma): 2NF + nincs **transzitiv** függőség. Transzitiv akkor van, ha egy nem-kulcs attribútum meghatároz egy másik nem-kulcsot — pl. `diak_id → varos_id → varos_nev`. Ilyenkor szétbontjuk a táblát.
10. A **BCNF** (Boyce-Codd normálforma) a 3NF szigorúbb változata: **minden funkcionális függőség (FF) bal oldala szuperkulcs** kell legyen — ezt akkor sérted, ha egy nem-szuperkulcs attribútum meghatároz egy másikat. Gyakorlatban a 3NF/BCNF szint a célzott — magasabb (4NF, 5NF) ritkán szükséges.

Történet

Az adatbázis-modellek a strukturált adatok rendszerezett tárolásának különböző paradigmáit jelentik. Az **hierarchikus modell** (régibb IBM IMS) fa-szerkezetben tárol adatot: minden rekordnak egyetlen szülője van. Egyszerű, de a sok-sok kapcsolatot nem tudja természetesen kifejezni, és redundanciát okoz. A **hálós modell** (CODASYL) ezen úgy lép túl, hogy egy rekordnak több szülője is lehet — így az M:N kapcsolat is kezelhető, de a navigáció bonyolult mutatókkal. Ezeket a modelleket az **relációs modell** váltotta fel (Codd, 1970), ami táblákban tárolja az adatot, és a kapcsolatokat **kulcsokkal** és **idegen kulcsokkal** fejezi ki — halmazalapú lekérdezéssel (SQL). A relációs modellben a kulcsoknak több típusa van. A **szuperkulcs** olyan attribútum-halmaz, amelytől az összes többi attribútum egyértelműen meghatározott; egy táblának sok szuperkulcsa lehet. A **candidate (kulcsjelölt)** egy minimális szuperkulcs — ha bármit elveszünk belőle, már nem szuperkulcs. A **primary key (PK)** az egyik kiválasztott candidate, kötelezően egyedi és NOT NULL, táblánként csak egy van. A **foreign key**

(FK) másik tábla PK-jára mutat, és ez az eszköz, amivel a kapcsolatokat kifejezzük. A kapcsolatok háromfélék lehetnek: **1:1** (egy A-hoz egy B, pl. személy-útleve), **1:N** (egy A-hoz több B, pl. osztály-tanuló), és **M:N** (több A-hoz több B, pl. hallgató-tantárgy), utóbbit relációsban kapcsolótáblával valósítjuk meg. Normalizálatlan táblákban **anomáliák** lépnek fel: beszúrási (új adat csak másikkal együtt vihető be), módosítási (egy adat több helyen, nehéz konzisztensen frissíteni), törlési (más adat törlése értékes infót veszít el). A **funkcionális függőség** azt mondja: ha $X \rightarrow Y$, akkor amennyiben két sorban X megegyezik, Y is megegyezik. A függőségek **transzitívak**: ha $X \rightarrow Y$ és $Y \rightarrow Z$, akkor $X \rightarrow Z$ is. A normalizálás célja ezeknek az anomáliáknak a megszüntetése: **1NF** szerint minden cella atomi (nem összetett, nem ismétlődő); **2NF** szerint minden nem-kulcs attribútum teljesen függ az összes elsődleges kulcstól; **3NF** szerint nincs tranzitív függés a nem-kulcsok között; a **BCNF** ennek szigorúbb változata, ahol minden funkcionális függőség bal oldala szuperkulcs. A gyakorlatban a 3NF/BCNF a célzott szint.

3.b Dinamikus adatszerkezetek

1. A hatékonyságot **algoritmizálási** (paradigma, korai kilépés) és **adatkonstruksiós** (cache-barát, hely-idő tradeoff) döntések befolyásolják.
2. A **verem (Stack)** LIFO szerkezet **push/pop/peek** műveletekkel; használat: hívási lánc, kifejezés-kiértékelés, undo.
3. A **sor (Queue)** FIFO szerkezet **enqueue/dequeue** -val; használat: ütemezés, BFS gráfbejárás, üzenetsorok.
4. A **láncolt lista** csomópontokból + mutatókból áll: beszúrás/törlés ismert pozíción $O(1)$, hozzáférés index szerint $O(n)$ — szemben a tömbbel.
5. A **hash-tábla** hash-függvénnyel képez kulcsot indexre, átlag $O(1)$ műveletek; ütközéskezelés **láncolással** vagy **nyílt címzéssel**.
6. **Kereső algoritmusok**: lineáris $O(n)$, bináris (rendezett) $O(\log n)$, hash $O(1)$, kiegyensúlyozott fa $O(\log n)$.
7. A **programozási tételek** átültethetők: tömbön index-iteráció, láncolt listán mutatóval, fán bejárások keretében.
8. A **rekurzió** olyan algoritmus, ami önmagát hívja egy egyszerűbb részfeladatra; két része van: a **báziseset** (kilépési feltétel — triviálisan megoldható eset, ahol megáll) és a **rekurzív hívás** (kisebb részfeladatra). Klasszikus példák: faktoriális, Fibonacci, Hanoi tornyai, fa-bejárások.
9. **Rekurzió vs iteráció**: a rekurzió a hívási vermet használja (stack overflow!), az iteráció konstans memóriájú — minden rekurzív átírható iteratívra.
10. A **fa** csomópontokból áll, a tetején a gyökérrel; a **BST** (Binary Search Tree, bináris keresőfa) olyan bináris fa, ahol minden csomópont **bal részfája < csomópont < jobb részfája** — kiegyensúlyozottan $O(\log n)$ keresés. **Bejárások**: preorder (csomópont → bal → jobb), inorder (bal → csomópont → jobb — BST-n **rendezett** kimenet), postorder (bal → jobb → csomópont), és **BFS** (Breadth-First Search, szélességi bejárás — szintenként, sorral implementálva).

Történet

A dinamikus adatszerkezetek és a kapcsolódó algoritmusok témakör a hatékonyság szempontjaival kezdődik. Az algoritmizálás során fontos a megfelelő paradigma kiválasztása (iteratív, rekurzív, dinamikus programozás, oszd-meg-és-uralkodj), a korai kilépés és a redundáns számítások elkerülése. Az adatkonstruksióánál a megfelelő adatszerkezet választása, a memória-lokalitás (cache-barát hozzáférés) és a hely-idő tradeoff a kulcs. Az aszimptotikus jelölés (O , Θ , Ω) segít leírni a műveletek skálázódását. A **verem (Stack)** LIFO (Last In, First Out) szerkezet **push**, **pop**, **peek** műveletekkel; alkalmazása a függvényhívási lánc, a kifejezések kiértékelése (postfix), a zárójelek ellenőrzése és az undo. A **sor (Queue)** FIFO (First In, First Out) szerkezet **enqueue** és **dequeue** műveletekkel; körkörös tömbbel vagy láncolt listával implementálható, alkalmazása az ütemezés, a szélességi gráfbejárás (BFS) és az üzenetsorok. A **láncolt lista** csomópontokból áll, amelyek mutatókkal kötődnek egymáshoz: lehet egy- vagy kétirányú; a beszúrás/törlés ismert pozíción $O(1)$, de a hozzáférés index szerint $O(n)$ — szemben a tömbbel, ahol a hozzáférés $O(1)$, de a közbülső beszúrás $O(n)$. A **hash-tábla** egy hash-

függvénnyel képez kulcsot indexre, és átlagosan $O(1)$ műveleteket biztosít; ütközéskezelése láncolás vagy nyílt címzés (lineáris/négyzetes próba, dupla hash); ha a load factor túl nagy, újraépítjük (rehashing). Kereső algoritmusok: a **lineáris keresés** $O(n)$ rendezetlen sorozaton, a **bináris keresés** $O(\log n)$ rendezett sorozaton (a felezést használja), a **hash-keresés** átlag $O(1)$, a **fa-keresés** $O(\log n)$ kiegyensúlyozott bináris keresőfán. A programozási tételek átültethetők bármely homogén adatszerkezetre. A **rekurzió** olyan algoritmus, ami önmagát hívja egy egyszerűbb részfeladattal; két része kell legyen: a **báziseset** (kilépési feltétel, ahol triviálisan visszaadja az eredményt) és a **rekurzív hívás** (kisebb részfeladatra). A rekurzió a hívási vermet használja, ezért stack overflow-ba futhat; az iteráció konstans memóriájú — minden rekurzív algoritmus átírható iteratívra explicit veremmel. A **fa** csomópontokból áll, a tetején egyetlen gyökérrel; a bináris fában minden csomópontnak legfeljebb két gyermeke van, és a **bináris keresőfa (BST)** olyan, hogy minden csomópont bal részfája kisebb értékeket, a jobb részfája nagyobbakat tartalmaz. A kiegyensúlyozott BST-n (AVL, piros-fekete) a műveletek $O(\log n)$ -ben futnak. A fa bejárásai: **preorder** (csomópont $\rightarrow$ bal $\rightarrow$ jobb), **inorder** (bal $\rightarrow$ csomópont $\rightarrow$ jobb, BST-n rendezett kimenetet ad), **postorder** (bal $\rightarrow$ jobb $\rightarrow$ csomópont), és a szélességi bejárás **BFS** szintenként, sorral implementálva.

4.a Magasszintű programozási nyelvek I — alapok

- 1. Primitív típusok:** byte/short/int (32)/long (64), float/double/decimal, bool, char (Unicode), és a `string` mint immutable referenciatípus.
- 2. A változó** memóriaterület + név + típus; a `const` fordításkor rögzül, a `readonly` csak konstruktorban kaphat értéket, a `literál` beégetett érték.
- 3. Operátorok:** aritmetikai, összehasonlító, logikai rövidzárú (`&&` , `||`), bitenkénti, ternáris, null-coalescing (`??`).
- 4. Szelekciós szerkezetek:** `if/else if/else` , `switch` (modern C#-ban pattern matchinggel).
- 5. Ciklusok:** `while` (elől), `do-while` (hátul), `for` (számláló), `foreach` (IEnumerable-n); vezérlés: `break` , `continue` , `return` .
- 6. Értéktípusok** (struct, int) a `stack`-en, érték szerint másolódnak; **referenciatípusok** (class) a `heap`-en, csak referenciát másolunk.
- 7. A stack** gyors LIFO és korlátos méretű (stack overflow); a `heap` dinamikus, a `GC` kezeli; a `boxing/unboxing` lassú, kerülendő.
- 8. A változó hatásköre** mondja meg, hol látszik (block, metódus, osztály), az **élettartama** azt, mikor él (lokális, mező, static).
- 9. Nyelvgenerációk:** 1GL gépi kód, 2GL assembly, 3GL magas szintű (C, C#, Java), 4GL DSL (SQL), 5GL logikai (Prolog).
- 10. Paradigma-családok:** imperatív (hogyan), funkcionális (függvény-kompozíció, immutable), deklaratív (mit — SQL), objektum-orientált, logikai.

Történet

A magasszintű nyelvek alapjai a típusoktól, változóktól indulnak. A **primitív típusok** közé tartoznak az egész típusok (byte, short, int 32 bit, long 64 bit, és előjel nélküli változataik), a lebegőpontos típusok (float 32, double 64, decimal 128 bit a pénzügyi számításokhoz), a logikai `bool` , a karakteres `char` (Unicode, 16 bit), és a szöveges `string` (referenciatípus, de érték-szemantikás és immutable). A **változó** egy névvel ellátott memóriaterület, ami értékek tárolására szolgál és értéke változhat. A **konstans** értéke a fordításkor rögzül és futás közben nem változhat, a **readonly** változó csak konstruktorban kaphat értéket, a **literál** pedig egy közvetlenül a kódba írt érték. Operátorok közül vannak aritmetikaiak (+, -, , /, %), összehasonlítókat (=, !=, <, >), logikai rövidzárúak (`&&` , `||` , `!`), bitenkéntiek, értékadókat, ternárisakat (`?:`), és modern C#-ban null-coalescing (`??`). A szelekciós szerkezetek az `if/else if/else` és a `switch` (modern C#-ban pattern matchinggel). A ciklusok lehetnek `while` (előltesztelő), `do-while` (hátteltesztelő, legalább egyszer fut), `for` (számláló), és `foreach` (kollekciók elemein, IEnumerable interfészen). A vezérlést `break` , `continue` és `return` befolyásolja. Az értéktípusok (struct, int, double, enum) a stack-en élnek és érték szerint másolódnak; a referenciatípusok (class, string, array) a heap-en élnek, és a változó csak referenciát tartalmaz rájuk. A stack gyors, LIFO, de korlátos méretű (stack overflow); a heap nagy, dinamikus, és a GC (Garbage Collector) automatikusan felszabadítja a már nem hivatkozott objektumokat. A boxing/unboxing

(értéktípus átalakítása object-té és vissza) lassú, kerülendő. A változó hatásköre mondja meg, hol látszik: block, metódus, osztály vagy namespace szintű. Az élettartam azt szabja meg, mikor él a változó: a lokális változók a stack frame-mel együtt szűnnek meg, a mezők az objektum élete alatt élnek, a static tagok az alkalmazás teljes futása alatt. A nyelvek generációi öt szintet különböztetnek meg: 1GL a gépi kód, 2GL az assembly, 3GL a magas szintű hardverfüggetlen nyelvek (C, C#, Java), 4GL a problématerület-specifikus nyelvek (SQL, MATLAB), 5GL pedig a logikai programozási nyelvek (Prolog). A nyelveket paradigma szerint is csoportosítjuk: imperatív (lépésről lépésre, hogyan), funkcionális (függvénykompozíció, immutable adatok), deklaratív (mit akarunk, nem hogyan — pl. SQL), objektum-orientált (osztályok, polimorfizmus), és logikai* (szabályok és tények, az interpreter következtet).

4.b Operációs rendszerek

1. Az operációs rendszer a **hardver és az alkalmazások közötti réteg**, ami erőforrást menedzsel és API-t nyújt.
2. A **kernel** privilegizált módban fut és közvetlenül a hardverrel beszél — lehet **monolitikus** (Linux), **mikrokernel** (Minix), vagy **hibrid** (Windows NT).
3. A **processz** saját címtérrel + PID-del + állapottal rendelkezik; a **szál** processzen belüli, közös memóriájú végrehajtási egység gyors kontextusváltással.
4. A **virtualizáció** lehet teljes (külön OS, hypervisor), para- (vendég tud róla), vagy **OS-szintű** (konténer, közös kernel — Docker).
5. **Fájlrendszerek** hierarchikus szervezést, attribútumokat, ACL-eket, lock-olást, **journalingot** nyújtanak — NTFS, ext4, APFS, ZFS, FAT32.
6. **RAID** szintek: 0 (striping, nincs redundancia), 1 (mirror), 5 (striping + 1 paritás), 6 (2 paritás), 10 (mirror + stripe).
7. **Átírányítások** (`>` , `>>` , `<` , `|`) és **szűrők** (`grep` , `sed` , `awk` , `sort` , `uniq`) láncolnak parancsokat Unix-filozófiában.
8. **Jogosultságok**: Linuxon owner/group/others × r/w/x (`chmod 755`); Windowson részletes **ACL** Allow/Deny szabályokkal.
9. **Processz-állapotok**: new → ready → running → waiting → terminated; ütemezők: FCFS, SJF, Round Robin, prioritás, multilevel feedback.
10. **Szignálok** (SIGTERM, SIGKILL, SIGINT) aszinkron értesítések; **backup**: full/inkrementális/differenciális (3-2-1 szabály); **shell-script** automatizál parancsokat shebanggel és vezérléssel.

Történet

Az operációs rendszer a hardver és az alkalmazások közötti közvetítő réteg: erőforrást menedzsel és API-t nyújt a programoknak. A rendszer magját a **kernel** alkotja, ami privilegizált módban fut és közvetlenül a hardverrel kommunikál; lehet monolitikus (a Linux minden szolgáltatása a kernelben), mikrokernel (csak a minimum, mint a Minix vagy QNX), vagy hibrid (Windows NT, macOS). A **processz** egy futó program egy példánya, saját memóriatérrel, állapottal, fájlleíróval és PID-del. A **szál (thread)** egy processzen belüli könnyűsúlyú végrehajtási egység, ami a többi szállal közös memóriát használ, gyors kontextusváltással. A felhasználói módban a programok korlátozott jogokkal futnak, kernel módban teljes hozzáféréssel. A **virtualizáció** lehetővé teszi, hogy egy fizikai gépen több izolált környezet fusson: a teljes virtualizációban VM-ek futnak külön OS-szel hypervisor felügyelete alatt; a para-virtualizációban a vendég OS tudja, hogy virtuális; az OS-szintű virtualizációban (konténerek, Docker, LXC) közös kernelt használnak, izolált processz-tér mellett. A **fájlrendszer** hierarchikus szervezést, jogosultságot, lock-olást és **journalingot** biztosít — utóbbi crash után gyorsan konzisztens állapotot ad. Tipikus rendszerek: NTFS (Windows, journaling+ACL), ext4 (Linux, gyors), APFS (macOS, SSD-optimalizált, snapshot), ZFS és Btrfs (modern, integrált snapshot), FAT32 (kompatibilis, USB). A háttértár szervezésénél a partícionálás MBR (régi, max 2TB) vagy GPT (modern) sémát követ; a **RAID**

különböző hibatűrési szintjei közül a RAID 0 a striping (sebesség, redundancia nélkül), a RAID 1 a mirroring (tükrözés), a RAID 5 striping plusz egy paritás (1 lemez-hibát tűr), a RAID 6 plusz két paritás, a RAID 10 pedig mirror plusz stripe. Az **átírányítások** (`>` , `>>` , `<` , `|`) és a **szűrők** (`grep` , `sed` , `awk` , `sort` , `uniq` , `wc`) láncolnak parancsokat Unix-filozófiásan. A **jogosultsági rendszer** Linuxon owner/group/others csoportokra és read/write/execute műveletekre bontja a fájljogokat (chmod 755 = rwxr-xr-x), Windowson pedig részletes ACL-eket használ Allow/Deny szabályokkal, örökölhetően. A **processz-állapotok** ciklusa: new → ready → running → waiting (I/O-ra) → terminated; az **ütemező** dönt, melyik processz fut (FCFS, SJF, Round Robin időszellettal, prioritás-alapú, multilevel feedback). A **szignálok** aszinkron értesítések: SIGTERM kérés a leállásra, SIGKILL azonnali leállítás (nem fogható el), SIGINT a Ctrl+C, SIGSEGV memóriasértés, SIGCHLD gyerek befejeződött. Az **adatmentés** lehet teljes (full), inkrementális (utolsó óta változott), differenciális (utolsó full óta változott); a 3-2-1 szabály szerint 3 másolat, 2 különböző médián, 1 off-site. A **shell-scriptek** parancssori parancsok sorozatát automatizálják, `#!/bin/bash` shebanggel, változókkal, vezérlési szerkezetekkel és exit kóddal.

5.a OOP alapok

1. Az **OOP négy alapelve**: egységbe zárás, adatrejtés, öröklődés, polimorfizmus.
2. Az **osztály** sablon az objektumokhoz: **mezők** az adatot tárolják, **metódusok** a viselkedést, a **konstruktor** az inicializálást.
3. **Láthatóság-szabályozók**: **private** (csak az osztály), **public**, **protected** (+ leszármazottak), **internal** (assembly-n belül).
4. A **property** C# specifikus: getter/setter mögött metódus fut, ezért validáció és invariáns érvényesíthető hozzáféréskor.
5. **Konténerosztályok** a **System.Collections.Generic** -ben: **List<T>**, **Dictionary<K,V>**, **Stack<T>**, **Queue<T>**, **HashSet<T>** — mind generikus, típusbiztos.
6. Az **indexelő** (**this[int i]**) lehetővé teszi, hogy az objektum **tömbszerűen indexelhető** legyen saját getter/setterrel.
7. **Static (osztályszintű)** tag az osztályhoz tartozik (Math.Sqrt), nem példányhoz; a **static konstruktor** egyszer fut, az osztály első használatakor.
8. **Névterek (namespace)** logikailag csoportosítják a típusokat és kerülik a névütközést — pl. **System.IO**, **System.Linq**.
9. **Bővítő metódus**: static osztály static metódusa, első paraméter elé **this** — meglévő típushoz adunk metódust forrásmódosítás nélkül; a LINQ erre épül.
10. **Operator overloading**: saját operátorok definiálhatók (csak static); az **==** és **≠** párban, és ha **==** -t definiálsz, **Equals** és **GetHashCode** is felülbírálandó.

Történet

Az objektum-orientált programozás (OOP) négy alapelve épül. Az **egységbe zárás** (encapsulation) szerint az adat és a rajta értelmezett műveletek egy egységben, az osztályban kapnak helyet. Az **adatrejtés** (information hiding) szerint az osztály belső állapota kívülről nem közvetlenül elérhető, csak publikus metódusokon és property-ken keresztül, ami biztosítja a belső invariánsokat. Az **öröklődés** (inheritance) új osztály létrehozását teszi lehetővé egy meglévő alapján, ami az ősnévképességeit öröklí és kibővítheti. A **többalakúság** (polimorfizmus) szerint ugyanaz a metódusnév más-más viselkedést jelenthet az osztály konkrét típusától függően. Az osztály egy sablon, amiből objektum-példányokat hozunk létre; a konstruktor inicializálja az új objektumot. A **mezők** az adatot tárolják, és láthatóságuk lehet **private** (csak az osztály), **public** (bárhonnan), **protected** (osztály + leszármazottak), **internal** (assembly-n belül), **static** (osztályszint) vagy **readonly** (csak konstruktorban módosítható). A **metódusok** a viselkedést definiálják, szignatúrával hívódnak, és túlterhelhetők. A C# specifikus **property** kívülről mezőként viselkedik, de mögötte metódushívás történik (getter/setter), ami lehetővé teszi a validációt és invariáns-érvényesítést. Az adatrejtéssel együtt ez biztosítja, hogy a belső állapot mindig konzisztens. A **System.Collections.Generic** névtér adja a beépített **konténerosztályokat**: **List<T>** dinamikus tömb, **Dictionary<K,V>** hash-tábla, **HashSet<T>** halmaz, **Stack<T>** és **Queue<T>** verem és sor, **LinkedList<T>** láncolt lista —

mind generikus, típusbiztos és boxing nélküli. Az **indexelő** (`this[int i]`) lehetővé teszi, hogy az osztály objektuma tömbszerűen indexelhető legyen, saját getter/setterrel. **Példányszintű** tagok az objektumhoz tartoznak, példányonként külön értékkel; a **static (osztálysintű)** tagok az osztályhoz, egyetlen közös példányban, példányosítás nélkül elérhetők (pl. `Math.Sqrt(2)`). A static konstruktor egyszer fut le, automatikusan, az osztály első használata előtt. A **névterek** (namespace) logikailag csoportosítják a típusokat és kerülik a névütközéseket — a BCL is így szervezett (System, System.IO, System.Collections.Generic). A **bővítő metódusok** statikus osztály statikus metódusai, ahol az első paraméter elé `this` kerül; ezzel meglévő típushoz adhatunk új metódust forrásmódosítás nélkül — a LINQ teljes egészében erre épül. Az **operátor-túlterhelés** saját operátorok definiálását teszi lehetővé osztályokhoz: csak `static` lehet, az `==` és `≠` valamint a `<` és `>` párban definiálandók, és ha `==` -t adunk meg, az `Equals` és `GetHashCode` is felülbírálendő.

5.b Architektúrák — cache + I/O

1. A **cache** kis, gyors átmeneti tároló a CPU és a memória közt — **cache hit** esetén nincs memóriához fordulás, **miss** esetén várni kell.
2. A cache működésének alapja a **lokalitási elv: időbeli** (a most használt adatot újra használjuk) és **térbeli** (a szomszéd is jön).
3. **Memória-hierarchia**: regiszter → L1 (32-64 KB, 3-5 ciklus) → L2 → L3 → RAM (100-200 ciklus) → SSD/HDD; a cache a CPU és a RAM közé ékelődik.
4. A cache **sorokra (cache line, ~64 bájt)** van osztva: tag (cím), adatblokk, **valid bit**, **dirty bit**.
5. **Elérési változatok**: direkt leképezésű (1 hely, gyors, ütközhet), fully associative (bárhova, drága), n-way set associative (n hely, kompromisszum).
6. A **cím három részre** bomlik: **tag** (blokkot azonosít), **index** (melyik sor/halmaz), **offset** (blokkon belüli bájt).
7. **Write-through**: minden módosítás azonnal a memóriába is megy (egyszerű, lassú); **write-back**: csak cache-be + dirty bit, sor kiváltásakor írunk vissza.
8. A **valid bit** jelzi, hogy a sor adata érvényes-e (induláskor 0); a **dirty bit** jelzi, hogy módosult-e (kiváltás előtt visszaírás kell).
9. **I/O módok**: programmed I/O (polling, CPU-igényes), interrupt-vezérelt (periféria szól), **DMA** (közvetlen memória-periféria adatmozgás, CPU csak indít).
10. **Perifériák**: PC-n USB billentyűzet/monitor/SSD; mobilon érintőképernyő, accelerométer, GPS, kamera; robotikán LIDAR, IMU, enkóder + GPIO/PWM/I²C/SPI vezérlés.

Történet

A számítógépek teljesítményét nagyban meghatározza a memória-hierarchia és a cache. A **cache** egy kis méretű, gyors átmeneti tároló a CPU és a lassabb fő memória között; célja az átlagos memória-elérési idő csökkentése. Ha a kért adat a cache-ben van, **cache hit** történik (gyors); ha nincs, **cache miss**, és a lassabb memóriához kell fordulni. A cache hatékonyságának alapja a **lokalitási elv**: az **időbeli lokalitás** szerint a CPU hajlamos ugyanazt az adatot rövid időn belül újra használni (pl. ciklusváltozók), a **térbeli lokalitás** szerint pedig a hozzáfért memóriaterület környezete is valószínűleg használatba kerül (pl. tömb-bejárás). Ezért a cache **adatblokkokat** (cache line, jellemzően 64 bájt) húz be egyszerre. A memória-hierarchiában a leggyorsabb a CPU **regiszterek** (1 ciklus), majd az **L1 cache** (32-64 KB, 3-5 ciklus), az **L2** (256 KB – 1 MB, 10-20 ciklus), az **L3** (4-32 MB, magok közös, 30-50 ciklus), a **RAM** (GB-os, 100-200 ciklus), és végül a háttértár (SSD/HDD, milliszekundumos). A cache **sorokra** van osztva; egy sor tartalma: **tag** (a memóriabeli eredeti helyet azonosítja), **adatblokk**, **valid bit** (érvényes-e a sor adata), **dirty bit** (módosult-e). Az elérési változatoknál a **direkt leképezésű** cache esetén minden memóriablokk pontosan egy cache-sorba kerülhet — gyors, de ütközhet; a **fully associative** cache-ben bármely blokk bárhova kerülhet — drága az ellenőrzés; a **n-way set associative** köztes megoldás: a cache halmazokra van osztva, és egy blokk n helyre kerülhet egy halmazon belül. A memóriacím három részre bomlik: **tag** (azonosítja a blokkot), **index** (melyik

halmaz), és **offset** (blokkon belüli bájt-pozíció). Az írási módoknál a **write-through** azonnal a memóriába is ír (egyszerű, de lassú), míg a **write-back** csak a cache-be ír és a dirty bittel jelöli — kiváltáskor írja vissza a memóriába. A **valid bit** jelzi, hogy a sor érvényes-e (bekapcsoláskor 0), a **dirty bit** pedig, hogy módosítva lett-e (akkor visszairás szükséges). Az **I/O** témakörben a CPU és a perifériák közti adatcsere három módon történhet: **polling** (a CPU rendszeresen lekérdezi a periféria állapotát, egyszerű de CPU-pazarló), **interrupt-vezérelt** (a periféria szól, a CPU mást csinálhat addig), és **DMA (Direct Memory Access)** — egy dedikált vezérlő közvetlenül a memória és a periféria között mozgat adatot, a CPU csak indítja és értesítést kap a végén. PC-n tipikus perifériák a billentyűzet, egér, monitor (HDMI, DisplayPort), nyomtató, SSD (NVMe, SATA), hálózati eszközök (Ethernet, Wi-Fi). Mobil eszközökön az érintőképernyő mellett gazdag szenzor-park található: gyorsulásmérő (orientáció), giroszkóp (forgási sebesség), magnetométer (iránytű), GPS (helymeghatározás), közelség-érzékelő, fényérzékelő, ujjlenyomat-olvasó. Robotikai környezetben szenzor-aktuátor rendszerek dolgoznak: LIDAR a lézeres térképezéshez, ultrahangos (sonar) közelség-detektálás, IR szenzorok, kamera computer vision-höz, IMU (gyorsulásmérő + giroszkóp + magnetométer együtt), enkóderek a motor-pozícióhoz; aktuátorok közt DC motorok, léptető, szervók, és ezek vezérlése GPIO digitális ki/be jelekkel, PWM motorvezérléssel, valamint soros buszokkal (I²C, SPI, UART, CAN).

6.a Numerikus matematika

1. **Hibák négy típusa:** kiküszöbölhetetlen input-hibák, modell-hiba, módszer-hiba, gépi (kerekítési) hibák.
2. **Abszolút hiba** $|a - a_0|$, **relatív hiba** $(a - a_0)/a_0$, **hibaterjedés** $\Delta y \approx |f'(x)| \cdot \Delta x$ — a relatív hiba és a biztos számjegyek között logaritmikus kapcsolat van.
3. **Nemlineáris egyenletekre** ($f(x) = 0$) az **intervallumfelező eljárás** biztos, de lassú; feltétele: $f(a) \cdot f(b) < 0$ előjelváltás.
4. A **Newton-Raphson** érintő-módszer **kvadratikusan konvergenciát** ad: $x_{n+1} = x_n - f(x_n)/f'(x_n)$; deriválhatóság és jó kezdőérték kell.
5. **Numerikus integrálás:** téglalap-módszer nulladfokú, **trapéz** elsőfokú, **Simpson** másodfokú interpoláción alapul — egyre pontosabb.
6. A **Simpson-formula** súlyozása **1-4-2-4-2- ... -4-1** mintát ad, $(h/3)$ szorzóval — feltétele páros n .
7. **Lineáris egyenletrendszereket Gauss-eliminációval** oldunk: kiterjesztett mátrixot felső háromszögre alakítjuk elemi sorrelációval átalakításokkal, visszahelyettesítünk.
8. **Polinomok kiértékelésére** a **Horner-módszer** szorzás-összeadássá rendezi a kifejezést (nincs hatványozás); a **Taylor-polinom** egy a pont körül polinomi sorral közelít.
9. **Interpoláció:** ismert pontokra polinomot illeszt — a **Lagrange** minden új pontnál újraszámol, a **Newton-féle (osztott differenciás)** inkrementális, csak az új tagot adjuk hozzá.
10. A **legkisebb négyzetek módszere** pontthalmazhoz a legjobban illeszkedő egyenest keresi az eltérések négyzetösszegét minimalizálva — nem interpoláció.

Történet

A numerikus matematika célja, hogy olyan matematikai problémákra adjon közelítő megoldást, amelyekre a számítógép közvetlenül nem alkalmas — például végtelen sorok kiszámítására, vagy valós számok pontos kezelésére. A megoldás hibákkal terhelt; négy hibatípust különböztetünk meg: **kiküszöbölhetetlen** input-hibák (mérés hibák), **modell-hiba** (a matematikai modell csak közelíti a valóságot), **módszer-hiba** (a numerikus eljárás pontatlansága), és **számítógépes** (kerekítési) hibák. Az **abszolút hiba** $|a - a_0|$, ahol a_0 a pontos érték; a **relatív hiba** ennek hányadosa a pontos értékkel — százalékban kifejezve. A **hibaterjedés** során a származtatott mennyiség hibája $\Delta y \approx |f'(x)| \cdot \Delta x$ alakú. A nemlineáris egyenleteket ($f(x) = 0$) több módszerrel oldhatjuk meg. Az **intervallumfelező eljárás** mindig konvergál, ha f folytonos és előjelet vált $[a, b]$ -ben ($f(a) \cdot f(b) < 0$): folyamatosan felezzük az intervallumot, és azzal a féllal folytatjuk, ahol a zérushely van. A **húrmódszer** két ponton áthaladó egyenes metszéspontját veszi az x tengellyel — gyorsabb, mint a felezés. A **Newton-Raphson** módszer csak egy kezdőpontot igényel, és az f érintőjének metszéspontját veszi az x tengellyel: $x_{n+1} = x_n - f(x_n)/f'(x_n)$; **kvadratikusan konvergenciát** ad (gyors), de deriválhatóság kell, és rossz kezdőponttal divergálhat. A **szelő módszer** nem deriválható függvényénél is működik, a Newtonnál lassabb. Az **inverz interpoláció (fixpont-iteráció)** során $f(x) = 0$ -t átírjuk $x = g(x)$ alakra és iterálunk: $x_{n+1} = g(x_n)$, konvergencia, ha $|g'(x)| < 1$ a fixpont környékén. A **numerikus integrálás** közelíti a görbe alatti területet:

az $[a, b]$ intervallumot n részre osztjuk, és minden részen egy közelítő alakzat területét összegezzük. A **téglalap-módszer** nulladfokú interpoláció, a **trapéz-módszer** elsőfokú, a **Simpson-formula** másodfokú (Lagrange interpoláción alapul); ez utóbbi súlyozása $1-4-2-4- \dots -4-1$ mintát ad, és $(h/3)$ szorzóval kapjuk az integrál közelítését — feltétele a páros n . A **lineáris egyenletrendszereket** ($A \cdot x = b$) **Gauss-eliminációval** oldjuk: a kiterjesztett mátrixot $[A \mid b]$ elemi sorkvivalens átalakításokkal (sor-csere, szorzás nem-nulla számmal, hozzáadás másik sorhoz) felső háromszögmátrixra hozzuk, majd visszahelyettesítünk. A polinomok kiértékelésére a **Horner-módszer** a hatványokat szorzás-összeadássá rendezi át, így nincs hatványozás, csak n szorzás és n összeadás. A **Taylor-polinom** egy a pont körül polinomi sorral közelíti a függvényt: $f(x) \approx f(a) + f'(a)(x-a) + \frac{f''(a)(x-a)^2}{2!} + \dots$; minél több tagot veszünk, annál pontosabb. Az **interpoláció** ismert pontokra olyan polinomot illeszt, ami átmegy mindegyiken; a polinom fokszáma a pontok száma $- 1$. A **Lagrange-interpoláció** alappontonként egy bázispolinomot ad, és ezeket összegzi; hátránya, hogy új pont hozzáadásakor mindent újra kell számolni. A **Newton-féle (osztott differenciás) interpoláció** ezt javítja: inkrementális — új pont hozzáadásakor csak egy új tagot kell hozzáadni. A **legkisebb négyzetek módszere** (lineáris regresszió) pontthalmazhoz a legjobban illeszkedő egyenest keresi az eltérések négyzetösszegét minimalizálva; ez nem interpoláció, csak közelítés, és feltételezi, hogy a változók között van kapcsolat.

6.b Operációs rendszerek — funkciók

1. Az OS **fő funkciói**: erőforrás-menedzsment, processz/szál kezelés, memóriakezelés, fájlrendszer, I/O, biztonság, hálózat, felhasználói felület.
2. A **kernel** privilegizált módban fut, a processzek user módban; a kontextusváltás közöttük **system call**-okon át történik.
3. A **processz** saját címtérrel + PID-del rendelkezik; a **szál** processzen belüli, közös memóriájú végrehajtási egység gyors váltással.
4. **Ütemezési algoritmusok**: FCFS, SJF, Round Robin (időszelet), prioritás-alapú, multilevel feedback queue — méltányosság és válaszidő közti tradeoff.
5. **Virtualizáció**: teljes (külön OS hypervisorral) vagy **konténer** (közös kernel, izolált processz-tér, pl. Docker).
6. A **fájlrendszer** hierarchikus szervezést, jogosultság-vezérlést, lock-olást, **journalingot** nyújt — utóbbi crash után konzisztens állapotot ad.
7. **Tipikus fájlrendszerek**: NTFS (Windows), ext4 (Linux), APFS (macOS), ZFS/Btrfs (modern, snapshot), FAT32 (kompatibilitás).
8. **Hibatűrő diszk-rendszerek (RAID)**: RAID 1 mirror, RAID 5 striping + 1 paritás (1 lemez-hibát tűr), RAID 6 (2), RAID 10 (mirror + stripe).
9. **Jogosultsági rendszer**: **autentikáció** (ki vagyok) + **autorizáció** (mit tehetek); Linuxon UID/GID + bitek, Windowson ACL.
10. **Backup szintek**: full (minden), inkrementális (utolsó óta), differenciális (utolsó full óta) — a **3-2-1 szabály** szerint 3 másolat, 2 médium, 1 off-site.

Történet

Az operációs rendszer fő funkciói nyolc fő területre oszthatók: erőforrás-menedzsment (CPU, memória, I/O, fájlok elosztása), processz- és szálkezelés (ütemezés, IPC, szinkronizáció), memóriakezelés (virtuális memória, lapozás, swap), fájlrendszer (absztrakció a háttértár felett), I/O kezelés (driverek, perifériák), biztonság és jogosultság (felhasználók, csoportok, ACL-ek), hálózati szolgáltatás (TCP/IP stack, socketek), valamint felhasználói felület (shell, GUI). A rendszer magja a **kernel**, ami privilegizált módban fut és közvetlenül a hardverrel kommunikál; a processzek user módban futnak, és a két szint közti átmenet **system call**-okon át történik. A **processz** önálló futási egység saját címtérrel, PID-del, állapottal és fájlleíróval; a **szál** processzen belüli, közös memóriájú, könnyűsúlyú végrehajtási egység, amely között a kontextusváltás gyors (csak regiszterek). Szinkronizációhoz mutex, szemafor, kondíciós változó használható. Az **ütemező** (scheduler) dönt, melyik processz/szál fut: a **FCFS** (First Come First Served) egyszerű, de konvoj-hatást okozhat; a **SJF** (Shortest Job First) optimális várakozási időt ad, de éhezetheti a hosszú feladatokat; a **Round Robin** időszelet-alapú; a **prioritás-alapú** ütemezés rangsorra épít; a **multilevel feedback queue** több prioritási sort használ dinamikus áthelyezéssel. A **virtualizáció** lehet teljes (külön OS-szel, hypervisorral) vagy OS-szintű (konténer, közös kernellel) — utóbbi gyors és könnyű (Docker). A **fájlrendszer** hierarchikus szervezést, attribútumokat, ACL-eket, lock-olást és **journalingot** nyújt; a

journaling crash után gyorsan visszaállít konzisztens állapotot. Tipikus rendszerek: NTFS (Windows, ACL, kompresszió, titkosítás), ext4 (Linux, gyors, nagy lemezekre), APFS (macOS, copy-on-write, snapshot), ZFS és Btrfs (modern, snapshot, integrált RAID), FAT32 (kompatibilis). A **hibatűrő diszk rendszerek (RAID)** különböző stratégiákat kínálnak: RAID 0 striping (sebesség, redundancia nélkül), RAID 1 mirror (tükrözés, redundancia), RAID 5 striping plusz egy paritás (egy lemez-hibát tűr, gazdaságos), RAID 6 két paritással (két lemez-hibát tűr), RAID 10 mirror plusz stripe (gyors és redundáns); az OS gyakran szoftveres RAID-et is támogat (Linux mdadm, Windows Storage Spaces). A **jogosultsági rendszer** két fő tevékenysége az **autentikáció** (ki vagyok — jelszó, token, biometria) és az **autorizáció** (mit tehetek). Linuxon UID/GID alapú a rendszer, 3 csoport (owner, group, others) × 3 művelet (r, w, x) bitekkel, és a sudo ad ideiglenes magasabb jogosultságot. Windowson részletes ACL-ek vannak felhasználónként/csoportonként, Allow/Deny szabályokkal, örökölhetően, és UAC kezeli az ideiglenes emelést. Mindkét rendszer audit-ol (ki mit csinált). A backup és archiválás stratégiái (full, inkrementális, differenciális, 3-2-1 szabály) a 4.b tételben tárgyalattal megegyeznek.

7.a OOP — öröklődés

1. Az **öröklődés** új osztály egy meglévő alapján: a leszármazott öröklí a védett és publikus tagokat, kibővítheti, felülbírálhatja.
2. C#-ban **egyszeres osztály-öröklődés** + tetszőleges interface implementálható; a **sealed** kulcsszó megtiltja a további öröklődést.
3. **Korai kötés** fordításkor a deklarált típus alapján; **késői kötés** futáskor az objektum tényleges típusa alapján — **virtual** + **override** kell hozzá.
4. A **virtual** metódusokhoz az osztály egy **DMT-t (vtable)** épít, és ebből oldódik fel a hívás — ez működteti a polimorfizmust.
5. A **konstruktor** az osztály nevével megegyezik, nincs visszatérési típusa; lehet több (overload), és **: this(...)** szintaxissal hívható másik saját konstruktor.
6. Származtatott osztály létrehozásakor **először az ős konstruktora fut le**, majd a leszármazotté — explicit **: base(...)** hívással irányítható.
7. A **static (osztálszintű) konstruktor** egyszer fut, automatikusan, az osztály első használata előtt — a static mezőket inicializálja.
8. **Típuskompatibilitás**: leszármazott implicit **upcastolható** ősre; **downcast** explicit cast vagy **as / is** operátorral, és kockázatos.
9. Az **object** minden típus végső őse — 4 örökölt metódusa: **ToString**, **Equals**, **GetHashCode**, **GetType**.
10. A **statikus osztály** nem példányosítható (csak static tagok, pl. Math); a **dynamic** kulcsszó (DLR-rel) futáskor oldja fel a típusokat — rugalmas, de elveszti a fordítóidejű biztonságot.

Történet

Az **öröklődés** lehetővé teszi, hogy új osztályt egy meglévő alapján hozzunk létre, ami az ős védett és publikus tagjait öröklí, és kibővítheti vagy felülbírálhatja őket. C#-ban egyszeres osztály-öröklődés van (egy osztálynak csak egy ős-osztálya lehet), de tetszőleges számú interface implementálható; a **sealed** kulcsszóval megtiltható a további öröklődés. A metódushívás feloldása kétféle lehet: a **korai (statikus) kötés** során a fordító a változó deklarált típusa alapján dönt fordításkor; a **késői (dinamikus) kötés** során a hívás futás közben, az objektum tényleges típusa alapján oldódik fel — ez működteti a polimorfizmust. A C#-ban alapból korai kötés van; a **virtual** és **override** kombóval érhető el a késői kötés. A virtual metódusokhoz az osztály egy **DMT-t** (Dinamikus Metódus Tábla, vtable) épít, ami az ős felé mutat, és a hívás ebből oldódik fel. A **new** módosító elrejtí az ős metódusát kötéssel, és ekkor nincs polimorfizmus — a deklarált típus dönt. A **konstruktor** speciális metódus, ami az objektum létrehozásakor fut le; az osztály nevével azonos, nincs visszatérési típusa, és lehet több túlterhelt változata. Ha nem írunk konstruktort, a fordító alapértelmezetten generál egy paraméter nélkülit. A **: this(args)** szintaxissal egy másik saját konstruktor hívható ugyanazon osztályból. Származtatott osztály létrehozásakor mindig **először az ős konstruktora fut le**, majd a leszármazotté; a **: base(args)** szintaxissal explicit hívható az ős-konstruktor. Az **osztálszintű**

(static) konstruktor egyszer fut, paraméter nélkül, automatikusan az osztály első használata előtt — a static mezők inicializálását végzi, és nem hívható manuálisan. A típusok közti konverzió két irányban történhet: **felfelé castolás (upcast)** a leszármazott típusú objektum hozzárendelése ősi típusú változóhoz, ami implicit és mindig biztonságos; **lefelé castolás (downcast)** az ősből leszármazottba, ami explicit cast-tal vagy az `as` / `is` operátorral történik, és kockázatos (futásidejű kivétel, ha nem az a típus). Az **object osztály** minden C# típus végső őse, és négy fontos örökölt metódussal jön: `ToString()` (szöveges reprezentáció), `Equals(object)` (egyenlőség), `GetHashCode()` (hash kód), `GetType()` (futásidejű típus) — ezek felülbírálnak. A **statikus osztály** (`static class`) nem példányosítható, csak static tagokat tartalmazhat, és implicit sealed; hasznos matematikai/segédfüggvények csoportosítására (pl. Math, Console). A **dinamikus osztály** koncepció a `dynamic` kulcsszóval valósul meg C#-ban: a fordító nem ellenőrzi a műveleteket, és minden a futás során a **DLR (Dynamic Language Runtime)**-on keresztül oldódik fel. Hasznos COM-interopnál, JSON/XML dinamikus feldolgozásánál, és reflection alternatívájaként; az `ExpandoObject` futás közben bővíthető objektum. A dinamikus vs statikus típusosság szemszögéből C#, Java, C++, Rust statikus (típusok fordításkor rögzítettek), míg Python, JavaScript, Ruby dinamikus (futás közben oldódnak fel).

7.b Adatbázis I — SQL

1. Az **SQL** szabványos, deklaratív lekérdező nyelv relációs adatbázisokhoz; al-nyelvei a **DDL** (séma), **DML** (adatok), **DCL** (jogok), **TCL** (tranzakció).
2. **Relációsémát** `CREATE TABLE` -l definiálunk; megszorítások: `PRIMARY KEY`, `FOREIGN KEY ... REFERENCES`, `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`.
3. Az **index** B-fa alapú segédstruktúra, ami **felgyorsítja a keresést** — gyors SELECT, lassabb INSERT/UPDATE; a PK automatikusan indexelt.
4. **Táblák módosítása**: `ALTER TABLE ADD/MODIFY/DROP`, `DROP TABLE`.
5. A **SELECT alapforma**: `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`; a logikai kiértékelés sorrendje FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
6. **Aggregáló függvények**: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`; a `HAVING` csoportokon, a `WHERE` egyedi sorokon szűr.
7. **JOIN típusok**: INNER (csak párosak), LEFT (bal min., jobban NULL), RIGHT, FULL, CROSS (Descartes-szorzat) — egyenlőségi feltétellel.
8. **Beágyazott lekérdezés (subquery)**: skalár, `IN` / `NOT IN`, `EXISTS`, korrelált — a belső kérdés a külső sor egy oszlopára hivatkozik.
9. **GRANT** és **REVOKE** adja és veszi vissza a privilégiumokat; a **role** (`CREATE ROLE`) jogosultság-csoport, felhasználókhöz rendelhető.
10. A **tranzakció ACID tulajdonságokkal** rendelkezik (atomosság, konzisztencia, izoláció, tartósság); `COMMIT` véglegesít, `ROLLBACK` visszagörget, **DDL implicit COMMIT-tal** jár.

Történet

Az **SQL** (Structured Query Language) a relációs adatbázisok szabványos, deklaratív lekérdező és manipuláló nyelve — a felhasználó azt mondja meg, mit szeretne, nem azt, hogyan. Az SQL négy al-nyelvre tagolódik: a **DDL (Data Definition Language)** a sémák és táblák definiálására való (`CREATE`, `ALTER`, `DROP`); a **DML (Data Manipulation Language)** az adatok kezelésére (`INSERT`, `UPDATE`, `DELETE`, `SELECT`); a **DCL (Data Control Language)** a jogosultságok kezelésére (`GRANT`, `REVOKE`); a **TCL (Transaction Control)** a tranzakciókezelésre (`COMMIT`, `ROLLBACK`, `SAVEPOINT`). Egy relációsémát `CREATE TABLE` -l definiálunk, megszorításokkal: `PRIMARY KEY` az elsődleges kulcs, `FOREIGN KEY ... REFERENCES` az idegen kulcs, `NOT NULL` kötelező érték, `UNIQUE` egyediség, `CHECK` érték-megszorítás, `DEFAULT` alapérték. Az **index** különálló adatszerkezet (általában B-fa vagy B+-fa), ami felgyorsítja a WHERE keresést, JOIN-t és rendezést; ellenértékként lassítja az INSERT/UPDATE/DELETE műveleteket, mivel az indexet is karban kell tartani. A PRIMARY KEY és a UNIQUE oszlopok automatikusan indexelve. Táblák szerkezetét utólag `ALTER TABLE` paranccsal módosíthatjuk (ADD oszlop, MODIFY típus, DROP COLUMN), és `DROP TABLE` -l törölhetjük. A **SELECT parancs** alapformája: `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT n`. A logikai végrehajtási sorrend (nem a szintaktikai): FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. Az **aggregáló függvények** (`COUNT`, `SUM`, `AVG`, `MIN`,

MAX) csoportszintű értéket adnak; a **GROUP BY** csoportosít, a **HAVING** a csoportokon szűr (a WHERE a sorokon). A **DISTINCT** eltávolítja a duplikátumokat. Több táblára vonatkozó lekérdezésekhez **JOIN**-okat használunk: az **INNER JOIN** csak párosított sorokat ad vissza; a **LEFT JOIN** a bal tábla minden sorát, ahol nincs pár, NULL kerül; a **RIGHT JOIN** fordítva; a **FULL JOIN** mindkettő minden sorát; a **CROSS JOIN** Descartes-szorzatot ad. Self-join is lehetséges aliasszal. A **beágyazott lekérdezések** (subquery) egy SELECT belsejében másikat futtatnak: lehetnek skalárok (egyetlen érték), **IN** / **NOT IN** -esek (halmaz-tagság), **EXISTS** -esek (van-e legalább egy találat), vagy korreláltak (a belső a külső sor egy oszlopára hivatkozik). A **DML utasítások** közvetlenül kezelik az adatot: **INSERT INTO ... VALUES ...** , **UPDATE ... SET ... WHERE ...** , **DELETE FROM ... WHERE ...** . A **privilegiumok** kezelése a **GRANT** (adás) és **REVOKE** (visszavonás) parancsokkal történik; a **WITH GRANT OPTION** lehetővé teszi, hogy a felhasználó tovább is adhassa a jogot. A **szerepkörök** (**CREATE ROLE**) jogosultság-csoportok, amiket felhasználókhöz rendelhetünk — így a sok felhasználós jog-kezelés egyszerűsödik. A **tranzakciókezelés** alapja az **ACID** tulajdonságok: **Atomicity** (mindent vagy semmit), **Consistency** (a megszorítások a tranzakció előtt és után is teljesülnek), **Isolation** (a tranzakciók látszólag egymástól függetlenül futnak), **Durability** (commit után többé nem vész el). A **COMMIT** véglegesíti a változásokat, a **ROLLBACK** visszagörgeti őket; a **SAVEPOINT** köztes mentési pontot ad, ahova visszagörgethetünk. Implicit COMMIT történik DDL utasításnál vagy a felhasználó kilépésekor; implicit ROLLBACK, ha az alkalmazás rendellenesen fejeződik be.

8.a OOP haladó — abstract, generic, lambda

1. **Absztrakt metódus** (`abstract` , nincs törzs) még meg nem írt műveletet jelöl; ha van benne, az **osztály is abstract**, a gyerek kötelezően override-olja.
2. Az **interface** nyelvi elem (nem osztály), ami **szerződést** definiál — metódus- és property-szignatúrákat tartalmazhat, mezőt és konstruktort nem.
3. **Egyetlen ős-osztály + tetszőleges interface** — így C# megoldja a többszörös öröklődés problémáját; az interface más interface-ből származhat.
4. A **kivételkezelés filozófiája**: a program normál módban fut, hiba esetén hiba módba lép, **visszaugrál a hívási láncon**, ha senki sem javítja, leáll.
5. **try-catch-finally**: `try` a kritikus kód, `catch(Típus)` típus szerint kezel (több catch fentről le, speciális előrébb), `finally` mindig lefut — erőforrás-felszabadításhoz.
6. **Generikusok** típussal paraméterezhető osztályok/metódusok (`List<T>` , `Dictionary<K,V>`) — általános viselkedés definiálható kódDuplikáció nélkül.
7. A **delegate** metódusreferencia (funkciómutató); beépítettek: **Action** (eljárás), **Func** (függvény visszatéréssel), **Predicate** (1 paraméter, bool).
8. Az **event** több metódust köthet egy eseményhez, és hívásra mind lefut — az **anonim delegate** egyszeri callbackként használható.
9. A **lambda kifejezés** tömör függvény szintaxis (`n ⇒ n.Contains('i')`); a **LINQ** teljes egészében erre épül.
10. A LINQ bővítő metódusokat ad (`Where` , `Select` , `GroupBy` , `Join`) `IEnumerable`-re, és **összefűzhető láncolható** lekérdezéseket épít — hasonló mint a Java Streams-ben és Python comprehensionokban.

Történet

Az objektum-orientált programozás haladó eszközei az absztrakcióval kezdődnek. Az **absztrakt metódus** olyan metódus, amit még nem tudunk megírni, mert csak a gyerekosztályokban nyer értelmet — `abstract` kulcsszó jelöli, törzs nélkül. Ha egy osztálynak van absztrakt metódusa, az **osztály is abstract kell legyen**, és nem példányosítható; a gyerekosztály kötelezően override-olja az absztrakt metódusokat (kivéve ha az is abstract). Minden absztrakt metódus implicit `virtual` (késői kötésű). Property is lehet abstract. Az **interface** nyelvi elem, nem osztály: arra való, hogy egymással nem rokon osztályok funkcionalitását befolyásoljuk; megmondja, milyen **felületet** valósítson meg az implementáló osztály, de nem azt, hogyan. C#-ban metódus- és property-szignatúrákat tartalmazhat (mező, konstruktor, static elem tilos). Egy osztálynak csak egy ős-osztálya lehet, viszont tetszőleges számú interface-t implementálhat, és az interface más interface-ből származhat — így oldódik meg a **többszörös öröklődés**. Fontos BCL interface-ek: `IComparable` (`List.Sort` használja), `IEnumerator` és `IEnumerable` (`foreach`), `IList : ICollection` (`Add`, `Remove`, `Insert`). A **kivételkezelés** alapelvei: a program alapból normál módban fut, és nincs if-es hibakezelés szétszórva a kódban; ha hiba merül fel, a program hiba módba lép, visszaugrál a hívási

láncon a metódusokon át, és ha senki nem javítja, leáll. A kivételt a `throw` kulcsszóval dobjuk, és példányosítani kell hozzá egy `Exception` leszármazott típust (pl. `FileNotFoundException`, `DivideByZeroException`, `NotImplementedException`); saját kivételosztály is írható, ami `Exception` -ből származik és konstruktort kötelezően ír, az ő konstruktorát hívva. A kivételeket `try-catch-finally` blokkokkal kezeljük: a `try` blokkban van a kritikus kód, ami kivételt dobhat; a `catch(Típus)` blokk a megadott típusú kivételt elkapja és kezeli — több catch is lehet, fentről lefelé értékelődnek, ezért a speciálisabb kivételek előrébb kerüljenek; a `finally` blokk mindig lefut, függetlenül attól, hogy volt-e kivétel, és tipikusan erőforrás-felszabadításra szolgál. A blokkok egymásba ágyazhatók. A **generikusok** típussal paraméterezhető osztályokat és metódusokat tesznek lehetővé, ami nélkül minden típushoz külön osztály kellene, sok ismétléssel. Szintaxisa: `class OsztalyNev<T>`, és lehet több típusparaméter is. Beépített generikusok: `List<T>`, `Dictionary<K, V>`, `Stack<T>`, `Queue<T>`, `LinkedList<T>`. A **delegate** metódusreferencia — egy metódus belépési pontjára mutat, és C#-ban külön objektumként kezelt. Célja a 3-rétegű alkalmazás-fejlesztés támogatása: az alkalmazáslogika ne jelenítsen meg semmit direkt módon, hanem **callbacket** hívjon. Beépített generikus delegate-ek: az `Action<T1, T2, ... >` eljárást tárol (nincs visszatérési típus), a `Func<T1, T2, ..., K>` függvényt (K a visszatérési típus), a `Predicate<T>` pedig egy paraméter és bool visszatérés. Az **event** több függvényt köthet egy eseményhez — hívásakor mind lefut. Anonim delegate is használható egyszeri callbackként. A **lambda kifejezés** tömör függvénytípusad (`n => n.Contains('i')`), és a **LINQ (Language Integrated Query)** teljesen erre épül: bővítő metódusokat ad `IEnumerable`-re (`Where`, `Select`, `GroupBy`, `Join`, `Count`, `Sum`), amik összekapcsolhatók, láncolható lekérdezéseket építenek. Hasonló minta megtalálható Java-ban a Streams-szel és Python-ban a comprehensionokkal.

8.b Fordítóprogramok

1. A **fordítóprogram** forrásnyelvi szövegből tárgykódot állít elő; a **compiler** magas szintű kódot gépi kódra fordít, az **interpreter** értelmezi futás közben.
2. A **compiler felépítése**: input-handler → lexikális → szintaktikai → szemantikai elemző → belső reprezentáció → kódgenerátor → optimalizáló → kód-kezelő → tárgyprogram.
3. A **lexikális elemző** karaktersorozatból **tokeneket** állít elő és reguláris (Chomsky-3) nyelvet használ — egy **lexikális hiba** ismeretlen token (pl. `int 2ab`).
4. A **szimbólumtábla** a programban előforduló szimbólumok nevét és attribútumait (típus, cím, sorszám) tárolja — gyakran láncolt listával.
5. A **reguláris nyelvek (Chomsky-3)** szabályai `A → a` vagy `A → Ba` alakúak, egy irányban épülnek — lexikális elemzésre alkalmasak.
6. A **szintaktikai elemzők** eldöntik, hogy a program nyelvtanilag helyes-e, és **levezetési fát** állítanak elő.
7. A **rekurzív leszállás** minden szimbólumhoz eljárást rendel, amik sorban hívják egymást — egyszerű, de nem generálható, nem elterjedt.
8. Az **LR(k)** elemző bottom-up, az inputból indul a kezdőszimbólum felé; az **LL(k)** top-down, a kezdőszimbólumból indul, balról jobbra építi a fát.
9. A **táblázatos elemző** állapot-hármassal `(ay#, Xa#, v)` dolgozik: input maradéka, verem (mondatforma), alkalmazott szabályok — Első/Követő halmazok döntenek a helyettesítést.
10. A **szemantikai elemzés** statikus tulajdonságokat vizsgál (típuskompatibilitás, deklaráció, hatáskör, túlterhelés-egyértelműség), amik környezetfüggetlen nyelvtannal nem írhatók le.

Történet

A **fordítóprogramok** alapja, hogy forrásnyelvi szövegből tárgykódot állítanak elő, nyelvek közötti konverziót hajtva végre. A folyamat során a forrásprogram beolvasása után a fordító elvégzi a lexikális, szintaktikus és szemantikus elemzést, előállítja a szintaxisfát, generálja, majd optimalizálja a tárgykódot. Két alapvető megközelítés van: a **compiler** magas szintű forráskódot transzformál alacsony szintű nyelvre (assembly, gépi kód), gyakran lépcsőzetes folyamatban; az **interpreter** egy hardveres vagy virtuális gép, ami értelmezni képes a magasszintű nyelvet, lényegében egy kétszintű gép, amelynek az alsó szintje a hardver, a felső pedig az értelmező és futtató rendszer. A compiler felépítése sorban: forrás-kezelő (input-handler, ami beolvassa a forrást és levágja az újsor/CR karaktereket) → lexikális elemző → szintaktikai elemző → szemantikai elemző → belső reprezentáció → kódgenerátor → optimalizáló → kód-kezelő → tárgyprogram. A **lexikális elemző** bemenete a source handler kimenete (egy karaktersorozat), és feladata felismerni a nyelv lexikális elemeit (tokeneket), és helyettesíteni őket a szintaktikus elemző számára érthető jelekkel. A lexikális elemző reguláris (Chomsky-3) nyelvet használ. A **szimbólumtábla** tartalmazza a programszövegben előforduló szimbólumok nevét és jellemzőit; minden előfordulásnál keresés vagy beszúrás történik, gyakran láncolt listával implementálva. Egy szimbólumtábla-sor tartalma a szimbólum neve és attribútumai:

definíciós adatok (mit azonosít: változó, konstans, típus, eljárás; konstansnál érték, típusnál típusdescriptorra mutató pointer, eljárásnál paraméterek száma és típusa), típus, programbéli cím (a tárgyprogramba beépítendő hivatkozási cím), forrásnyelvi sorszám, hivatkozott sorszám és láncolási cím. Lexikális hiba akkor van, amikor az elemző nem ismer fel egy tokent (pl. `int 2ab` érvénytelen azonosító). A **reguláris nyelvek** (Chomsky-3 osztály) szabályai $A \rightarrow a$ és $A \rightarrow Ba$ (balreguláris) vagy $A \rightarrow a$ és $A \rightarrow aB$ (jobbreguláris) alakúak; a szó egy irányban épül fel, a köztes állapotokban csak egy nemterminális van mindig a jobb (vagy bal) végén. Nincs kitétel az $S \rightarrow \epsilon$ szabályra. Lexikális elemzésre alkalmas. A **szintaktikai elemzők** eldöntik, hogy az adott programozási nyelven írt program nyelvtanilag helyes-e, és feladatuk a jelsorozat levezetési fájának előállítás. A **rekurzív leszállás** során a grammatika minden szimbólumához eljárást rendelünk, amik egymást rekurzívan vagy egyszerű hívással hívják; a szalagon lévő adatot és a szabályt egyszerre lépteti. Egyszerű módszer, de a programnyelv implementációja valósítja meg az elemző vermét — nem lehet generálni, ezért nem elterjedt. Az **LR(k) elemző** bottom-up: az elemzendő szimbólumsorozatból indul ki, és a cél a kezdőszimbólum elérése — így építi fel a szintaxisfát alulról felfelé. Az **LL(k) elemző** top-down: a kezdőszimbólumból indul ki (ami a gyökér lesz), és a cél, hogy a szintaxisfa levelein az elemzendő szöveg terminálisai jelenjenek meg. A **táblázatos elemző** állapota egy $(ay\#, Xa\#, v)$ hármas: $ay\#$ a még nem elemzett szöveg végjellel, $Xa\#$ a verem tartalma (a mondatforma még nem elemzett része), v az alkalmazott szabályok sorszámainak listája. A kezdőállapot $(xay\#, S\#, \epsilon)$, a sikeres végállapot $(\#, \#, w)$. Az elemző mindig a verem tetején lévő szimbólumot hasonlítja a még nem elemzett szöveg következő szimbólumával, és az **Első/Követő halmazok** segítségével dönt a helyettesítésről. A **szemantikai elemzés** a szintaxisfa pontjaihoz attribútumokat rendel, amik leírják az adott pont tulajdonságait, és ezek konzisztenciáját vizsgálja. Olyan tulajdonságokat ellenőriz, amelyek statikusak és nem írhatók le környezetfüggetlen nyelvtannal: változók deklarációja, hatásköre, láthatósága, többszörös deklarációja, deklaráció hiánya; operátorok és operandusok közötti típuskompatibilitás; eljárások, tömbök formális és aktuális paraméterei közötti kompatibilitás; és túlterhelések egyértelműsége.

9.a Formális nyelvek

1. Az **ábécé** véges, nem üres jelhalmaz; a **szó** az ábécé jeleiből alkotott véges sorozat (ϵ az üres szó); a **formális nyelv** az ábécé fölötti szavak részhalmaza.
2. **Műveletek szavakkal: konkatenáció** (asszociatív, nem kommutatív, neutrális ϵ), hatványozás, tükrözés (palindrom); nyelvekkel: komplexus szorzás, hatvány.
3. **Szintaxisleíró eszközök**: a **szintaxisdiagram** vizuális (terminális/nemterminális téglalapok), az **EBNF** karakteres jelölés programnyelvekhez.
4. A **generatív grammatika** $G(V, W, S, P)$ formális négyes: V terminálisok, W nemterminálisok ($V \cap W = \emptyset$), S kezdőszimbólum, P szabályok.
5. A **Chomsky-hierarchia** 0-tól 3-ig szigorodik: 0 mondszerkezetű (Turing), 1 környezetfüggő (lin. korl. Turing), 2 környezetfüggetlen (verem-automata), 3 reguláris (véges automata).
6. A **levezetési fa** környezetfüggetlen nyelvtannal dönti el, hogy egy szó generálható-e — **kanonikus levezetésnél** mindig a legbaloldalibb nemterminálist helyettesítjük tovább.
7. A **véges automata** (K, V, δ, q_0, F) : nincs output, csak olvasó-fej; fix n lépés után megáll, akkor fogad el, ha elfogadó állapotban van.
8. A **verem-automata** $(K, V, W, \delta, q_0, z_0, F)$: van veremje, ahova szót írhat, δ -be ϵ is mehet input gyanánt — **nem biztos, hogy megáll**.
9. A **Turing-gép** $(K, V, W, \delta, q_0, B, F)$ az input szalagra is írhat, mindkét irányban végtelen; akkor áll meg, ha δ nincs értelmezve — változatai (több szalagú, lin. korlátolt) ekvivalensek.
10. **Automaták és grammatikák** kapcsolata: a grammatika **generál**, az automata **felismer** — a Chomsky-osztály és az automata-típus egy-egyértelműen összepárosul.

Történet

A **formális nyelvek** elmélete az ábécé és a szó fogalmával kezdődik. Az **ábécé** egy véges, nem üres halmaz, amelynek elemei jelek — karakterek, betűk, szimbólumok. Egy **szó** az ábécé jeleiből alkotott véges sorozat; az **üres szó** jele ϵ . Az ábécé fölött alkotható összes lehetséges szó halmaza a **lezárt V** (akár végtelen hosszú szavakkal), a pozitív lezárt V^+ ugyanez ϵ kivételével. Egy formális nyelv az ábécé fölötti szavak részhalmaza — vagyis egy nyelv egy adott ABC jeleiből alkotott tetszőleges hosszú szavak halmazának részhalmaza. Megadhatjuk felsorolással, szabály szöveges/matematikai leírásával, vagy generatív grammatikával. Műveletek szavakkal: a konkatenáció (jele $+$) két szóból egy újat képez egymás mellé illesztéssel; asszociatív, nem kommutatív, neutrális eleme ϵ . Egy szó n -edik hatványa az n -szeres konkatenációja, és a szó tükre a fordított sorrendű változat (palindrom: a tükre saját maga). Műveletek nyelvekkel (ábécékkel): a komplexus szorzás $A \cdot B := \{ab \mid a \in A, b \in B\}$; az n -edik hatvány A^n az n karakter hosszú szavak halmaza. A szintaxis-leírásnak két fő eszköze a szintaxisdiagram (vizuális, gömbölyű sarkú téglalapok a terminálisoknak, szögletesek a nemterminálisoknak) és az EBNF (Extended Backus-Naur Form, karakteres jelölés). A generatív grammatika célja, hogy szabályok alkalmazásával eldönthessük, egy szó eleme-e a nyelvnek. Formálisan egy $G(V, W, S, P)$ négyes: V a terminális jelek ábécéje, W a nemterminális jeleké ($V \cap W = \emptyset$), $S \in W$ a kezdőszimbólum, P a helyettesítési szabályok

halmaza. A szógenerálást a kezdőszimbólumtól kezdjük, és addig folytatjuk, amíg vannak nemterminális jelek. A Chomsky-féle osztályozás négy szigorodó nyelvosztályt különböztet meg. A 0. mondszerkezetű nyelvek szabályai $\alpha\beta \rightarrow \gamma$ alakúak — élő, beszélt nyelvek, fordításuk erőforrásigényes. Az 1. környezetfüggő nyelvek $\alpha\beta \rightarrow \alpha\omega\beta$ alakú szabályokkal dolgoznak (megengedett az $S \rightarrow \epsilon$); szemantikai elemzésre alkalmasak. A 2. környezetfüggetlen nyelvek $A \rightarrow \omega$ alakú szabályokkal (a bal oldalon csak nemterminális); szintaktikai elemzésre — programozási nyelvek leírására valók. A 3. reguláris nyelvek $A \rightarrow a$ vagy $A \rightarrow Ba$ (balreguláris) / $A \rightarrow aB$ (jobbreguláris) szabályokkal — lexikális elemzésre alkalmasak; nincs kitétel az $S \rightarrow \epsilon$ szabályra. A szabályok 0-tól 3-ig szigorodnak, így egy reguláris nyelv beilleszthető a többibe. A levezetési fa környezetfüggetlen nyelvtannal eldönti, hogy egy szó generálható-e: ha balról jobbra, fentről lefelé olvasva megjelenik a szó, akkor generálható. A kanonikus levezetésnél mindig a legbaloldalibb nemterminális szimbólumot helyettesítjük tovább. A levezetés lehet top-down (a kezdőszimbólumból, fentről lefelé) vagy bottom-up (alulról felfelé, szabályok behelyettesítésével), és visszalépéses elemzéssel próbálkozhatunk ha sikertelen. Az automaták minden Chomsky-osztályhoz egy konstrukciót rendelnek, ami eldönti egy input szóról, hogy helyes-e (0-ásra nincs algoritmus). A véges automata $G(K, V, \delta, q_0, F)$ ötös: K a véges belső állapotok halmaza, V az input ABC, δ az állapotátmeneti függvény, q_0 a kezdőállapot, F az elfogadó állapotok. Nincs output szalag, az olvasó-fej a szalagon halad jobbra; megáll, ha a fej lelép, és akkor fogad el, ha elfogadó állapotban van — fix n lépés után megáll. A verem-automata $G(K, V, W, \delta, q_0, z_0, F)$ hetes: van veremje (W az output ABC, z_0 a verem-üres szimbólum), δ paramétere lehet ϵ (nem olvasás), és a verembe szót ír. Nem mindig olvas az input szalagról, ezért nem biztos, hogy megáll. A Turing-gép $G(K, V, W, \delta, q_0, B, F)$ szintén hetes, de a szalagra írni is tud, és a szalag mindkét irányban végtelen; δ parciális, az automata megáll, ha δ nincs értelmezve az aktuális (q, a) páron. Változatai (egyirányú végtelen szalagos, több szalagú, lineárisan korlátolt) ekvivalensek az alapossal — kivéve a lineárisan korlátoltat, ami csak az 1. Chomsky-osztályt ismeri fel. A delta leképezés megadása az egyes automatákban más-más; véges automatánál táblázat vagy állapotgráf $(q, a) \rightarrow q'$ alakkal; verem-automatánál $(q, a \text{ vagy } \epsilon, w_{\text{top}}) \rightarrow (q', w_{\text{write}})$; Turing-gépnél $(q, a) \rightarrow (q', a', \text{irány})$. Az automaták és grammatikák kapcsolata pontos megfeleltetés: egy nyelvet egy grammatika generál, és egy automata felismer* — az automata gyakorlatilag az algoritmus, amit a grammatikából csinálunk. A 0-ás Chomsky-osztálynak Turing-gép, az 1-esnek lineárisan korlátolt Turing-gép, a 2-esnek verem-automata, a 3-asnak véges automata felel meg.

9.b Rendszerfejlesztés technológiája

1. A szoftver **egyedi** (ismert megrendelő) vagy **dobozos** (ismeretlen vásárlók); az **életciklus** a módszertanok által meghatározott — cél a magas minőség minimális költséggel.
2. **9 állomás**: új igény → követelményspec → rendszerjavaslat → rendszerspecifikáció → logikai+fizikai tervezés → implementáció → tesztelés → rendszerátadás → üzemeltetés.
3. A **követelményspecifikáció** alapja **riportok** (szabad és irányított); tartalmazza a jelenlegi helyzetet, vágyálomrendszert, jogszabályi háttérrel, fogalomszótárt.
4. A **funkcionális specifikáció** felhasználói szemszögből írja le a rendszert, a **nagyvonalú rendszerterv** fejlesztői szemszögből.
5. Modern módszertanok **iteratívak**: a tervezés-implementáció-tesztelés ciklusok rövidek, minden iteráció szállíthat valamit.
6. A **feladatkövetés** (Kanban/Scrum board) átláthatóvá teszi a feladatokat — csökkenti az állapotjelentés szükségességét.
7. A **verziókövetés** (Git, SVN) a változásokat követi (ki, mikor, mit), támogat branchet, conflictnél értesít — a fejlesztés kulcseszköze.
8. **Tesztelési technikák**: feketedobozos (csak spec), fehérdobozos (forráskód), szürkedobozos (részleges) — kódsorokat, elágazásokat, metódusokat tesztelünk.
9. **Tesztelés szintjei**: unit (metódus), modul, integrációs, rendszerteszt, átadás-átvételi (alfa, béta, felhasználói, üzemeltetői).
10. A **regressziós teszt** minden változtatás után **az összes unit-tesztet** lefuttatja — biztosítja, hogy nem rontottuk el a már működőt.

Történet

A **rendszerfejlesztés technológiája** a szoftver életciklusát és a fejlesztés szervezett módját foglalja össze. A szoftver lehet **egyedi** (ismert megrendelőre készül, pl. állami megbízás) vagy **dobozos** (ismeretlen vásárlóknak, mint a játékok vagy operációs rendszerek). A szoftver élete folyamatos változás (új igények merülnek fel, bővíteni kell), ezért nevezzük ciklusnak; a lépéseket a választott módszertan határozza meg, és a cél a magas minőségű szoftver minimális költséggel. Az életciklus kilenc fő állomásból áll: új igény felmerülése a felhasználói oldalon → igények elemzése és követelményspecifikáció elkészítése → rendszerjavaslat kidolgozása (funkcionális specifikáció, ütemterv, szerződéskötés) → rendszerspecifikáció (megvalósíthatósági tanulmány, nagyvonalú rendszerterv) → logikai és fizikai tervezés → implementáció → tesztelés → rendszerátadás és bevezetés → üzemeltetés és karbantartás → új igény felmerülése. A **követelményspecifikáció** alapja a megrendelővel készített riportok, amik lehetnek szabadok (a megrendelő saját szavakkal mondja el az igényt, mi figyelünk és felhívjuk a figyelmét az ellentmondásokra) vagy irányítottak (általánosabb véleménykutatás cél, előre megírt kérdőívvel). A **rendszerjavaslat** fő része a funkcionális specifikáció (a felhasználó szemszögeiből), az ütemterv, az árajánlat és a szerződés. A **funkcionális specifikáció** a felhasználói szemszögből írja le a rendszert, és a követelményspecifikáció igényeire ad választ. A **nagyvonalú rendszerterv** a fejlesztői szemszögből készül, és tartalmazza a rendszer célját,

projekttervet, funkcionális tervet, fizikai környezetet, absztrakt domain modellt, architektúrális tervet, adatbázis tervet, implementációs tervet, tesztelési tervet, telepítési tervet és karbantartási tervet. A modern módszertanok **iteratíván** közelítenek: a tervezés-implementáció-tesztelés ciklusok rövidek, és minden iteráció szállít valamit. A **rendszerfejlesztés eszközei** közé tartozik a **feladatkövetés** (pl. Kanban-tábla vagy Scrum board), ami átláthatóvá teszi a projekt feladatait — a programozók kiválasztják, hogy mire dolgoznak, ha kész, áthelyezik a "kész" oszlopba, és így csökken a gyakori állapotjelentés szükségessége. A **verziókövetés** (Git, SVN) az állományok tartalmi változásait követi (ki és mikor módosította), korábbi állapotokat is elő tud hívni, állapotok közötti különbséget mutat (diff), két módosítás ütközése (conflict) esetén értesít, és elágazásokat (branch) tesz lehetővé. A **tesztelési technikák** három fő kategóriába esnek: a **feketedobozos** tesztelésnél csak a specifikáció ismert, és abból készülnek a tesztesetek; a **fehértobozos** tesztelésnél a forráskód alapján; a **szürkertobozos** tesztelésnél a forráskód egy része ismert. Strukturálisan kódsorokat, elágazásokat, metódusokat, osztályokat, funkciókat és modulokat tesztelünk. A **tesztelés szintjei** négy nagy területre oszlanak: a **komponensteszt** a rendszer komponenseit külön-külön teszteli (ezen belül unit-teszt a metódusokra, modulteszt a modulok használat szerinti tesztje); az **integrációs teszt** a komponensek közötti együttműködést; a **rendszer teszt** az egész rendszert együttesen, ellenőrizve, hogy megfelel-e a követelmény- és funkcionális specifikációknak (gyakran külön cég végzi); az **átadás-átvételi teszt** a végfelhasználók tesztje a kész rendszeren (alfa-teszt a cégnél nem-fejlesztőkkel, béta-teszt szűk felhasználói csoporttal, felhasználói átvételi teszt majdnem minden felhasználóval, üzemeltetői átvételi teszt a rendszergazdákkal). A **regressziós teszt** az összes unit-teszt lefuttatása egy változtatás után, hogy biztosítsa: a változás nem okozott hibát olyan metódusokban, amik korábban működtek.

10.a Hálózati architektúrák

1. A **csomagkapcsolt hálózatok** az üzenetet csomagokra bontják, mindegyik fejléccel megy, különböző útvonalakon, és a sorrendet a célnak kell helyreraknia.
2. Az **OSI 7 rétegű** elméleti modell: fizikai, adatkapcsolati, hálózati, szállítási, viszony, megjelenítési, alkalmazási — az oktatás referenciája.
3. A **TCP/IP 4 rétegű** gyakorlati modell: network access, internet (IP), transport (TCP/UDP), application — ez fut élesben az interneten.
4. A **router** útválasztási táblája alapján dönt; protokollok: statikus, **distance-vector** (RIP), **link-state** (OSPF, Dijkstra), **BGP** (autonóm rendszerek között).
5. Az **IPv4** 32 bites cím, hálózat-azonosító + host-azonosító; az **alhálózati maszk** (/24) szabja meg a felosztást; **privát tartományok**: 10.x, 172.16.x, 192.168.x.
6. Az **IPv6** 128 bites, beépített biztonsággal; a **NAT** privát címeket nyilvánosra fordít — IPv4-cím spórolásra.
7. A **TCP** kapcsolatorientált, megbízható, stream alapú: **3-way handshake**, ACK-nyugtázás, újraküldés, folyamszabályozás, torlódásvezérlés.
8. Az **UDP** kapcsolat nélküli, gyors, megbízhatatlan — VoIP, streaming, online játékok, **DNS**, DHCP.
9. Az **ICMP** hálózati réteg protokoll hibajelzésre és diagnosztikára: Echo Request/Reply (**ping**), Time Exceeded (**traceroute**), Destination Unreachable.
10. A **DNS** domain név → IP feloldó hierarchikus rendszer (root → TLD → authoritative); rekordtípusok A, AAAA, CNAME, MX, NS, TXT — UDP 53-as porton.

Történet

A számítógépes hálózatok alapfogalmait a csomagkapcsolt hálózatok adják. A **hálózat** egymással kapcsolatban lévő csomópontok összessége, ahol a **csomópont** önálló kommunikációra képes, saját hálózati címmel rendelkező eszköz (számítógép, router, nyomtató). A **csomagkapcsolt** hálózatokban nincs előre kiépített út a kommunikálni kívánó számítógépek között; az üzenetet maximális méretű adatsomagokra bontjuk, és ezeket önálló egységeként küldjük el. A csomag fejrésze tartalmazza a címzett(ek) címét, és az útválasztók (router-ek) különböző útvonalakon juttathatják el — ezért a beérkezési sorrend megváltozhat, és a csomag akár el is vesztet, ha a router korlátozott kapacitása miatt el kell dobni; a címzettnek gondoskodnia kell a helyes sorrendbe rakásról. A hálózati architektúrák két klasszikus referenciamodellje az OSI és a TCP/IP. Az **OSI (Open Systems Interconnection)** elméleti modell 7 réteget különböztet meg, alulról felfelé: 1. fizikai réteg (bit-átvitel, kábel, jel), 2. adatkapcsolati réteg (keret, MAC, hiba-detektálás, Ethernet), 3. hálózati réteg (csomagok, útvonal-választás, IP), 4. szállítási réteg (végpont-végpont, TCP, UDP), 5. viszony réteg (session-ök), 6. megjelenítési réteg (kódolás, titkosítás), 7. alkalmazási réteg (HTTP, DNS, FTP). A **TCP/IP modell** gyakorlati és 4-5 rétegű: network access (fizikai+adatkapcsolati), internet (IP, ICMP), transport (TCP, UDP), application (HTTP, DNS, FTP, SSH, SMTP). Az OSI elméleti referencia, oktatási célokra, szigorú réteg-elválasztással; a TCP/IP a valós internet protokollkészlete, lazább réteg-elválasztással, ahol a

megjelenítési és viszony réteg az alkalmazási rétegbe olvad. A **forgalomirányítás** során a router az útválasztási tábla alapján dönti el, melyik felé továbbít egy csomagot. A protokollok lehetnek statikusak (kézzel beállítottak) vagy dinamikusak: a **distance-vector** protokollok (mint a RIP) szomszédoktól tanulnak hop-szám alapján; a **link-state** protokollok (mint az OSPF) mindenki ismeri a teljes topológiát és Dijkstra-algoritmussal számolnak; a **BGP (Border Gateway Protocol)** az autonóm rendszerek közötti útválasztásra szolgál. Az **IPv4** 32 bites cím, négy oktett pontokkal elválasztva (pl. 192.168.1.1). A cím részei a hálózat-azonosító és a host-azonosító, és a felosztást az **alhálózati maszk** adja meg, pl. /24 = 255.255.255.0 = 24 bit hálózat + 8 bit host. Privát címtartományok (RFC 1918): 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. A **CIDR (Classless Inter-Domain Routing)** rugalmas alhálózati méreteket tesz lehetővé, és felváltotta a régi osztályos rendszert. Az **IPv6** 128 bites cím, sokkal több címmel, beépített biztonsággal és autokonfigurációval. A **NAT (Network Address Translation)** a belső privát címeket nyilvánosra fordítja, IPv4-cím spórolásra. A **TCP (Transmission Control Protocol)** kapcsolatorientált, megbízható, stream alapú szállítási protokoll: kapcsolatot 3-way handshake-kel épít fel (SYN → SYN-ACK → ACK), minden csomagot nyugtáz (ACK), újraküld ha nincs nyugta időben, sorszámozással helyreállítja a sorrendet, és folyam- valamint torlódásvezérlést alkalmaz. Felhasználása HTTP, HTTPS, SSH, FTP, SMTP — mindenhol, ahol fontos a megbízhatóság. Az **UDP (User Datagram Protocol)** kapcsolat nélküli, megbízhatatlan, csomag alapú; nincs handshake, nincs nyugtázás, nincs újraküldés — gyors, kis overhead, de a csomag elveszhet vagy sorrend cserélődhet. Felhasználása VoIP, streaming, online játékok, DNS, DHCP. Az **ICMP (Internet Control Message Protocol)** hálózati réteg protokoll, hibajelzésre és diagnosztikára: az **Echo Request/Reply** üzeneteket használja a ping; a **Destination Unreachable** jelzi, hogy a cél nem érhető el; a **Time Exceeded** üzeneten alapul a traceroute; a **Redirect** jobb útvonalat javasol. Az ICMP nem szállít user-adatot, csak kontroll- és hibajelzéseket. A **DNS (Domain Name System)** célja a domain név → IP cím feloldás (és vissza). Hierarchikus rendszer: a root server-ek (.) a tetején, alattuk a TLD server-ek (.com, .hu), majd az authoritative server-ek (specifikus domainekhez). Rekordtípusok: A (IPv4 cím), AAAA (IPv6), CNAME (alias), MX (mail szerver), NS (name server), TXT (szöveges). A feloldási folyamatban a kliens a resolver-t kérdezi, az a root-ot, majd a TLD-t, majd az authoritative szerveret, és cache-eli az eredményt. Általában UDP 53-as port, nagyobb válaszhoz TCP.

10.b Szolgáltatásorientált programozás

1. Az **RPC** távoli eljáráshívás: a kliens stub-bal marshallolja a paramétereket, hálózaton átküldi, szerver oldalon kicsomagolódik és visszatér.
2. A **gRPC** a Google modern RPC-je: HTTP/2 transzporton, bináris üzenetekkel, 4 streaming-móddal, többnyelvű kódgenerálással .proto fájlokból.
3. A **protocol buffers** a gRPC szerializációs nyelve — séma-alapú, bináris, kisebb és gyorsabb mint a JSON, és kompatibilis verziókezelést támogat.
4. A **WEB** decentralizált, HTTP-alapú; az **egyrétegű (monolit)** architektúrában minden egy alkalmazásban van.
5. A **többrétegű (3-tier)** felosztja a rendszert prezentáció (UI), üzleti logika, adat rétegre — minden réteg külön skálázható.
6. **Vékony kliens:** kevés a kliensen, sok szerveren (webalkalmazás); **vastag kliens:** sok kliens-oldali logika (asztali alkalmazás, SPA).
7. **Elosztott rendszerek** skálázhatóak (horizontálisan), hibatűrőek; a **CAP-tétel** szerint a Consistency-Availability-Partition tolerance hármasából csak 2 lehet.
8. A **REST** HTTP-alapú architektúráis stílus: **resource-orientált URI**, **stateless**, HTTP-igék (GET/POST/PUT/PATCH/DELETE), cache-elhető, JSON.
9. A **web service** hálózaton elérhető gép-gép API; **SOAP/XML** vagy **REST/JSON** — utóbbi ma az elterjedt.
10. A **WCF** Microsoft technológia szolgáltatásokhoz, **ABC** modellel (Address, Binding, Contract); az ASMX csak HTTP/SOAP, a WCF rugalmas és sokoldalú, de a modern alternatíva ASP.NET Core Web API vagy gRPC.

Történet

A **szolgáltatás-orientált programozás** a hálózaton keresztüli kommunikáció absztrakciójával foglalkozik. Az **RPC (Remote Procedure Call)** alapötlete, hogy a kliens úgy hív szerver-függvényt, mintha lokális lenne. A megvalósítás során a kliens **stub** csomagolja a hívást (paraméterek sorosítása, marshalling), átküldi a hálózaton, a szerver **stub** (skeleton) oldalon kicsomagolja, és hívja a tényleges függvényt; az eredmény visszamegy ugyanígy. Szinkron alapról, elrejt a hálózatot — könnyebb fejlesztés, de a hálózati hibák is rejtve maradnak. A modern változat a **gRPC (Google RPC)**: HTTP/2 transzporton fut (multiplexing, server push, fejléc-tömörítés), bináris üzeneteket használ (sokkal kisebb és gyorsabb, mint a JSON), és négyféle streaming-módot támogat (unary, server streaming, client streaming, bidirectional). A többnyelvű kódgenerálás **.proto** fájlokból történik. A **protocol buffers** a gRPC szerializációs nyelve: bináris, sémaalapú formátum, ami kisebb payload-ot és gyorsabb sorosítást ad, mint a JSON; erős típusos, és új mezők hozzáadása nem törli a régi klienseket (verziókezelés). A **WEB** decentralizált, HTTP protokoll alapú információs rendszer. Architektúráisan többféle elrendezést használunk. Az **egyrétegű (monolit)** alkalmazásban minden egy helyen van: UI, üzleti logika, adat. A **kliens-szerver modell** a logikát megosztja: a kliens kéri, a szerver szolgál. A **többrétegű (3-tier)** felépítésben szétválasztjuk a prezentációs réteget (UI, kliens), az üzleti logikát (alkalmazáserver) és az

adat réteget (DB), így minden réteg külön skálázható és karbantartható. **Vékony kliens** esetén keveset csinál a kliens, sok a szerveren (klasszikus webalkalmazás); **vastag kliens** esetén sok logika kerül kliens-oldalra (asztali alkalmazás, gazdag SPA). Az **elosztott rendszerek** több gépből állnak, ami egységes rendszerként viselkedik; jellemzői a horizontális skálázhatóság (több gép), a hibatűrés (egy gép kiesése nem dönti meg az egészet), és a hálózati latency. A **CAP-tétel** szerint a Consistency–Availability–Partition tolerance három tulajdonságból egyszerre csak kettő érhető el. A **REST (Representational State Transfer)** HTTP-alapú architekturális stílus. Alapelvei: erőforrás-orientált — URI azonosít egy erőforrást (pl. /users/123); **stateless** — minden kérés önálló, a szerver nem tárol kliens-állapotot; egységes interface — HTTP igéket használ (GET lekérdez, POST létrehoz, PUT teljes frissítés, PATCH részleges, DELETE töröl); cache-elhető válaszok; rétegzett architektúra (proxy-k, CDN-ek beékelhetők); reprezentáció jellemzően JSON. A **web service** hálózaton keresztül elérhető, gép-gép kommunikációs API. Két fő típus: **SOAP** XML-alapú, szigorú séma (WSDL), bonyolult; **REST** könnyebb, JSON-alapú, ma elterjedt. Mindkettő platformfüggetlen. A **WCF (Windows Communication Foundation)** Microsoft technológia szolgáltatás-orientált alkalmazásokhoz, a .NET keretrendszer része. Fő fogalmai az **ABC** modell: **Address** (hol érhető el a szolgáltatás, URI), **Binding** (hogyan kommunikál — HTTP, TCP, MSMQ, named pipe), **Contract** (mit kínál, interface, metódusok). A szolgáltatás (Service) a tényleges implementáció, az **Endpoint** az ABC kombináció, a **Host** a futtatókörnyezet (IIS, Windows Service, self-hosted), és a kliens (Client) proxy-val kommunikál, ami a contract alapján generálódik. A **régi ASMX** csak HTTP/SOAP-on dolgozott, XML szerializációval és egyszerű konfigurációval; a **WCF** több protokollt támogat (HTTP, TCP, MSMQ, NamedPipe), több szerializációt (XML, JSON, bináris), részletes konfigurációval és beépített WS-Security biztonsággal. Ma a Microsoft inkább az **ASP.NET Core Web API** vagy a **gRPC** felé tol mindenkit.

11.a Rendszerfejlesztés — módszertanok

1. Az **életciklus dokumentumai**: követelményspecifikáció, funkcionális specifikáció, nagyvonalú rendszerterv, tesztterv, átadás-átvételi jegyzőkönyv — a funkspec **megfeleltetést** is tartalmaz a kövspechez.
2. A **módszertanok** oszthatók: életciklus-sorrend (lineáris/iteratív), dokumentáltság (könnyű/nehézsúlyú), megközelítés (jól dokumentált, prototípus, agilis), középpont.
3. **Strukturált (hagyományos)** módszertanok lineárisak, nehézsúlyúak: **vízesés** nem enged visszatérést, a **V-modell** teszteléssel egészíti ki.
4. **Hagyományos vs agilis**: a hagyományos sok dokumentációt és merev fázisokat, az agilis működő kódot és rugalmas iterációkat, direkt kommunikációt preferál.
5. **Prototípus alapú megközelítés**: a felhasználó ne a projekt végén lássa először a terméket — **eldobható** (csak felmérésre) vagy **evolúciós** (a rendszer lelke lesz).
6. A **Scrum** agilis módszertan, 2-4 hetes **sprintekkel**: Sprint Planning, Daily Meeting, Sprint Review, és a legfontosabb **Retrospective**.
7. **Scrum szerepkörök**: **Scrum Master** (folyamat-felügyelő), **Product Owner** (priorizál, nem azonos SM-mel), **Team** (5-9 fő); a **Product Backlog** prioritizált sztorikat tartalmaz.
8. Az **extrém programozás (XP)** 4 tevékenységre (kódolás, tesztelés, odafigyelés, tervezés) és 5 értékre (kommunikáció, egyszerűség, visszacsatolás, bátorság, tisztelet) épül.
9. **XP technikák**: páros programozás, **TDD**, code review, folyamatos integráció, **refactoring** — akkor jó, ha a megrendelő tud átláthatóan együttműködni.
10. A **kockázatmenedzsment** két vetülete: bekövetkezés valószínűsége és kár mértéke; a **súlyosság** = **valószínűség × kár**, lépései: azonosítás, értékelés, csökkentés, kommunikáció (COBIT, Common Criteria).

Történet

A rendszerfejlesztés módszertanai határozzák meg, milyen sorrendben és módon haladunk a szoftverfejlesztés lépésein. Az életciklus dokumentumai közé tartozik a **követelményspecifikáció** (mit szeretne a megrendelő), a **funkcionális specifikáció** (felhasználói szemszögű válasz a kövspec-re, a megfeleltetést is tartalmazza — minden kövspec-beli követelményhez van-e funkció), a **nagyvonalú rendszerterv** (fejlesztői szemszög: mit, miért, hogyan, mikor, miből), a **tesztterv**, az **átadás-átvételi jegyzőkönyv**, és a **felhasználói dokumentáció**. A nagyvonalú rendszerterv részei: rendszer célja, projektterv, funkcionális terv, fizikai környezet, absztrakt domain modell, architekturális terv (MVC, rétegek, biztonság), adatbázis terv, implementációs terv, tesztterv, telepítési terv, karbantartási terv. A rendszerterv fajtái a konceptuális (mit, miért), a nagyvonalú (+ hogyan, miből) és a részletes (+ mikor). A módszertanok többféle szempont szerint oszthatók: az életciklus fázisok sorrendje (lineáris, spirális, iteratív); az implementációs nyelv preferenciája (folyamatorientált, adatközpontú, strukturált, OO, szerviz, keverék); a megközelítés (jól dokumentált, prototípus-alapú, rapid, agilis, extrém); a dokumentáltság (könnyűsúlyú vs nehézsúlyú); és a középpontban álló szempont (adat-, folyamat-, követelmény-, használati eset-, teszt-, felhasználó-, ember-, csapatközpontú). A **strukturált**

(**hagyományos**) **módszertanok** közé tartozik a vízésés, az SSADM és a V-modell. Ezek lineárisak, jól dokumentáltak, nehézsúlyúak, adat- és folyamatközpontúak, és nagy, hosszú projektekre valók. A **vízésés modell** lineáris: a lépések merev sorrendben követik egymást (szükségletek felmérése → rendszertervezés → megvalósítás → tesztelés → üzembe helyezés), és lezárt fázisba nem térhetünk vissza; rengeteg dokumentációt készít, amit elfogadtat a megrendelővel, hogy ne legyen félreértés. A **V-modell** a vízésés kiegészítése teszteléssel: a fejlesztési lépéseket tükrözik a tesztelési lépések, és ha hibát találunk, vissza kell menni az adott fejlesztési lépésre. További strukturált módszertanok a RUP, a Spirál modell, az Inkrementális fejlesztés, és a RAD (Rapid Application Development). A **hagyományos vs agilis** szembeállítás a dokumentáció (sok és részletes vs minimális), rugalmasság (alacsony vs magas), gyorsaság (lassú indulás vs gyors első kiadás) és kommunikáció (formális, írásos vs direkt, szóbeli) szempontjából lényeges. Az agilis résztvevők megpróbálnak amennyire lehet alkalmazkodni a projekthez. A **prototípus alapú megközelítés** célja, hogy a felhasználó ne a projekt végén lássa először a terméket, így a félreértések elkerülhetők. Több prototípust szállítunk a végső átadás előtt, és minden iteráció után pontosíthatjuk a követelményeket. Két fő típus: az **eldobható prototípus** csak a követelmények gyors felmérésére szolgál (papír alapú vagy mockup tool), és nem lesz része a végső programnak. Az **evolúciós prototípus** a fejlesztés elején készül, és a rendszer lelke lesz, amit később finomítunk — előnye, hogy az életciklus elején már működő rendszert kap a felhasználó. A **Scrum** agilis módszertan, ami a csapaton belüli összetartásra és a megrendelő változó igényeire épít. A **sprint** 2-4 hetes fejlesztési időszak, Sprint Planninggel kezdődik és Retrospective-vel zárul; minden sprint végén szállítható prototípus áll rendelkezésre. A **Daily Meeting**-en mindenki elmondja, mit csinált, mit fog, milyen akadályok merültek fel; a **Sprint Review**-n áttekintik az elkészülteket; a **Sprint Retrospective**-n pedig a felmerült problémákat (ez a legfontosabb a jövőbeli javulás miatt). Termékek: a **Product Backlog** prioritizált sztorikat tartalmaz, a **Sprint Backlog** az adott sprintre kiválasztottakat. A szerepek "disznók" (elkötelezettek) — **Scrum Master** (folyamat-felügyelő), **Product Owner** (priorizál, nem azonos SM-mel), **Team** (5-9 fő fejlesztő, tesztelő, elemző); és "csirkék" (érdekeltek) — üzleti szereplők, menedzsment. Az **extrém programozás (XP)** szintén agilis módszertan, ami a jól bevált technikákat veszi át. Négy tevékenységet ír elő: kódolás, tesztelés (extrém sok teszt, a tesztelés a dokumentáció), odafigyelés (megérteni a megrendelőt), tervezés (független komponensek, OOP). Öt értékre épít: kommunikáció, egyszerűség, visszacsatolás, bátorság, tisztelet. Technikái: páros programozás (egyik ír, másik figyel és szól ha hibát lát), TDD (teszt-vezérelt fejlesztés: előbb a teszt, aztán a metódus), code review, folyamatos integráció (nap végén integrációs teszt), és refactoring (működő kód szépítése, funkcionalitás megőrzésével). A **kockázatmenedzsment** célja, hogy felmérjük, mennyire sebezhető a rendszer, és ha kár történik, mekkora. Két vetülete a bekövetkezés valószínűsége és a kár mértéke, és a kockázat súlyossága ezek szorzata. Az eszközök között van a COBIT kockázatelemző és a Common Criteria biztonsági értékelés. A lépések: kockázat azonosítása, értékelése, csökkentése (pl. vírusirtó, banki tranzakció-biztonság), és kommunikációja.

11.b Nulladrendű logika

1. A **nulladrendű logika** ítéletváltozókat kapcsol logikai műveletekkel — minden objektum igaz/hamis; a **szintaxis** a helyes formulák nyelvtana, a **szemantika** az értelmezés.
2. Az **igazságtábla** segít összetett kifejezést kiértékelni; egy **interpretáció** minden atomhoz igaz/hamis értéket rendel.
3. **Szemantikai tulajdonságok**: **tautológia** (minden interpretációban igaz), **kontradikció** (mindig hamis), **kielégíthető** (legalább egyben igaz).
4. **Normálformák**: **literál** (atom vagy negáltja), **klóz** (literálok diszjunkciója), **KNF** (klózikonjunkciója), **DNF** (elemi konjunkciók diszjunkciója).
5. **KNF-re hozás**: ekvivalencia eltüntetése, implikáció $A \rightarrow B \equiv \neg A \vee B$, negációk bevitele atomokig (de Morgan), **disztributivitás**.
6. A **Tseitin transzformáció** lineáris algoritmus: minden nem-literál részformulához új X ítéletváltozót vezet be, és konjugál hozzá egy $X \leftrightarrow A$ ekvivalenciát.
7. A **Plaisted-Greenbaum kódolás** csökkenti a Tseitin klózszerát: az ekvivalencia két implikációra bontható, és a **polaritás függvényében** csak az egyik megtartandó.
8. A **rezolúció** cáfoló eljárás: a tagadásból az **üres klóz** levezetésével UNSAT-ot bizonyítunk — nulladrendűben **véges levezetés garantált**.
9. A **SAT** (KNF kielégíthetősége) **NP-teljes**; **3-SAT** is, **2-SAT** polinomiális; a **DPLL** visszalépéses keresés + **unit propagáció** az egységklózikra.
10. A **DIMACS** a SAT solver standard input formátuma (sorszámok, $-$ negálás, 0 klózvég); az **SMT** kiterjeszti aritmetikára/tömbökre/listákra, az **SMT-LIB** ennek emberközelibb formátuma.

Történet

A nulladrendű logika egyszerűbb mint az elsőrendű: minden objektuma igaz/hamis típusú, és logikai változókat (itéletváltozókat) kapcsolunk össze logikai műveletekkel — nem foglalkozunk az objektumok belső szerkezetével. Az elsőrendűben ezzel szemben tetszőleges típusú objektumokon foglalkozunk meg logikai állításokat predikátumokkal. A **szintaxis** azt írja le, hogyan lehet formailag helyes kifejezéseket csinálni a nyelven. Például "Ha Kristóf bejárt órákra és sikerült a dolgozata, akkor megkapja a kettést" formalizálva: legyen K = "Kristóf bejárt órákra", D = "sikerült a dolgozat", E = "kap kettést"; akkor $(K \wedge D) \rightarrow E$. Zárójelezéssel változtatható a kiértékelési sorrend, és formalizáláskor a lényegtelen részeket (igeidők) elhagyjuk. A **szemantika** a kifejezések jelentésével foglalkozik — a nulladrendűben az igazságértékükkel; ezt vizsgálja az igazságtábla. Az **igazságtábla** összetett kifejezések kiértékelésére jó: szabályokat fogalmazhatunk meg vele a konjunkcióhoz, implikációhoz stb. Egy **interpretáció** olyan i függvény, ami az F formula minden atomjához igaz vagy hamis értéket rendel — például az igazságtábla egy sora. Szemantikai tulajdonságok: egy formula **tautológia** (**logikai törvény**), ha minden interpretációban igaz; **kontradikció** (**logikai ellentmondás**, **kielégíthetetlen**), ha minden interpretációban hamis; **kielégíthető**, ha legalább egy interpretációban igaz. Egy formula **logikai következménye** egy másinak akkor, ha az $F_1 \wedge \dots \wedge F_k \wedge \neg C$ -ből

levezethető az üres klóz. A **normálformák** alapfogalmai: a **literál** egy atom vagy annak negáltja; a **klóz** literálok diszjunkciója; egy formula **konjunktív normálformában (KNF, CNF)** van, ha klózek konjunkciója; egy formula **diszjunktív normálformában (DNF)** van, ha elemi konjunkciók diszjunkciója. A **KNF-re hozás algoritmus**a négy fő lépésből áll: 1) az ekvivalencia eltüntetése: $A \leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$; 2) az implikáció eltüntetése: $A \rightarrow B \equiv \neg A \vee B$; 3) a negációk bevitele atomokig de Morgan azonosságokkal; 4) disztributivitás alkalmazása: $A \vee (B \wedge C) \rightarrow (A \vee B) \wedge (A \vee C)$. A probléma az, hogy a disztributivitás miatt **exponenciális méretű** lehet a klózhalmaz, mert vannak részformulák, amik duplikálódnak. Ezt küszöböli ki a **Tseitin transzformáció**, ami **lineáris algoritmus**: minden nem-literál részformulához bevezet egy új ítéletváltozót ($X_1, X_2, \dots$), a teljes A részformulát felülírja X -szel, és konjugál hozzá egy $X \leftrightarrow A$ ekvivalenciát. Kintről befelé érdemes haladni — először a teljes formulára X_1 -et, annak részformuláira X_2 -t, X_3 -at, és így tovább a literálok szintjéig. Az eredmény külső operátorai konjunkciók, innen már könnyű KNF-et csinálni. A **Plaisted-Greenbaum kódolás** tovább csökkenti a Tseitin által előállított klózek számát: az ekvivalencia két egymással szembe mutató implikáció, és ha az egyikről belátható, hogy szükségtelen, azzal spórolunk. A polaritás függvényében: ha X mindkét polaritással szerepel, akkor $X \leftrightarrow A$ marad; ha csak pozitív, akkor $X \rightarrow A$ elég; ha csak negatív, akkor $A \rightarrow X$. A polarítások vizsgálatakor feltételezzük, hogy az előtte lévő formula már KNF-ben van. A **rezolúció** nulladrendű logikában cáfoló eljárás: a bizonyítandó állítás tagadásából indulunk, és bemutatjuk, hogy levezethető az üres klóz (UNSAT). Az A formula tautológia, ha $\neg A$ -ból levezethető az üres klóz; $P_1 \dots P_k$ logikai következménye C , ha $P_1 \wedge \dots \wedge P_k \wedge \neg C$ -ből levezethető. A **rezolvens**: ha $C_1 = P \vee C'_1$ és $C_2 = \neg P \vee C'_2$, akkor $C'_1 \vee C'_2$ a rezolvensük (P és $\neg P$ kiesnek). Az algoritmus iteratív: válasszunk két még nem rezolvált klózt, állítsuk elő a rezolvensüket, ha az üres klóz előáll, UNSAT, állj, egyébként vissza az 1-re. Nulladrendűben véges levezetés garantált. A **SAT-probléma** azt kérdezi: kielégíthető-e egy KNF-ben adott formula? Ez **NP-teljes** probléma — nem ismert rá polinomiális algoritmus, csak exponenciális. A 3-SAT (3-hosszú klózek) is NP-teljes, de a 2-SAT-ra van polinomiális megoldó algoritmus. A **DPLL algoritmus** a sima rezolúciónál sokkal hatékonyabb: visszalépéses keresést használ, rekurzív, és kiemelten kezeli az egységklózeket (1 literált tartalmazókat). A **unit propagáció** a rezolvensképzés egy speciális esete: ha $C_1 = P$ egységklóz és $C_2 = \neg P \vee C'_2$, akkor a rezolvens C'_2 (rövidebb klóz). A **DIMACS** a SAT solverek standard input formátuma: minden ítéletváltozót sorszám jelöl, a negáció jele $\neg$, az elválasztó a szóköz, és a klóz végét 0 jelzi; a fejléc `p cnf <változó db> <klóz db>` formátumú. Az **SMT (Satisfiability Modulo Theories)** magasabb szintű feladatleírást ad: a nulladrendű logikát egész/valós számokkal, aritmetikával, tömbökkel, listákkal egészíti ki, és bizonyos elméletekre vonatkoztatva vizsgálja a kielégíthetőséget. Felhasználása szoftververifikáció, rendezések, logikai fejtörők, aritmetikai problémák. Az **SMT-LIB** az SMT solverek standard input formátuma — komplexebb, emberközelibb, mint a DIMACS: szöveges utasításokkal, típusokkal, aritmetikai operátorokkal és függvényhívásokkal.

12.a Adatbázis II — PL/SQL

1. A **PL/SQL** az Oracle procedurális kiterjesztése — változókkal, vezérléssel, kivételkezeléssel ágyazza be az SQL-utasításokat.
2. **Típusok**: skalár (NUMBER, VARCHAR2, DATE, BOOLEAN); **%TYPE** (oszlop típusa), **%ROWTYPE** (sor típusa), RECORD, TABLE (asszociatív tömb), VARRAY.
3. Változó deklarálása **név típus := kezdőérték**; **CONSTANT** konstanst, **NOT NULL** kötelező értéket ad.
4. **Vezérlési szerkezetek**: **IF/ELSIF/ELSE**, **CASE**, **LOOP-EXIT WHEN**, **WHILE**, **FOR i IN 1..10**, és cursor-FOR-loop ami automatikusan kezeli a kurzort.
5. **SQL beágyazása**: **SELECT INTO** egy sort egy változóba (NO_DATA_FOUND, TOO_MANY_ROWS); INSERT/UPDATE/DELETE közvetlenül; DDL csak **EXECUTE IMMEDIATE** -tel.
6. Egy **PL/SQL blokk** 3 részből áll: **DECLARE** (deklarációk), **BEGIN-END** (kötelező), **EXCEPTION** (opcionális) — blokkok egymásba ágyazhatók.
7. A **tárolt eljárás (PROCEDURE)** nem ad vissza értéket (csak OUT-on), önállóan hívható; a **tárolt függvény (FUNCTION)** **RETURN** -nel ad értéket és SELECT-ben is hívható.
8. A **csomag (PACKAGE)** logikailag összetartozó alprogramokat csoportosít: **specifikáció** (interface) és **body** (implementáció), encapsulation, memóriába egészében töltődik.
9. **Paraméter-módok**: **IN** (alapeset, csak olvasható), **OUT** (kimenet, null-ról indul), **IN OUT** (be- és kimenet) — pozicionális vagy megnevezett (**a ⇒ 1**) hívás.
10. **Kivételkezelés**: **WHEN** beépített kivétel (NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, OTHERS) vagy saját **EXCEPTION** + **RAISE**; **RAISE_APPLICATION_ERROR** -20000-től -20999-ig.

Történet

A **PL/SQL (Procedural Language for SQL)** az Oracle eljárás-szerű kiterjesztése az SQL-hez, ami lehetővé teszi, hogy SQL-utasításokat ágyazzunk procedurális kódba — változókkal, vezérlési szerkezetekkel, kivételkezeléssel együtt. A típusrendszer skalár típusokat (numerikus: NUMBER, INTEGER, PLS_INTEGER, FLOAT; karakteres: CHAR, VARCHAR2, CLOB; dátum: DATE, TIMESTAMP; logikai: BOOLEAN — csak PL/SQL-ben; bináris: RAW, BLOB) és összetett típusokat tartalmaz. A **%TYPE** átveszi egy tábla-oszlop típusát (pl. **nev Diakok.nev%TYPE**), a **%ROWTYPE** egy egész tábla sortípusát rekordként. Saját RECORD, asszociatív tömb (TABLE), VARRAY és NESTED TABLE típusok is definiálhatók. **Változót** a **név típus [:= kezdőérték]**; formával deklarálunk; a **CONSTANT** jelző azt jelenti, hogy futás közben nem módosítható, a **NOT NULL** pedig kötelező kezdőértéket kíván.

Vezérlési szerkezetek: szelekcióhoz **IF feltétel THEN ... ELIF ... ELSE ... END IF** és **CASE érték WHEN ... THEN ... ELSE ... END CASE**; ciklusokhoz **LOOP ... EXIT WHEN feltétel; END LOOP**, **WHILE feltétel LOOP ... END LOOP**, **FOR i IN 1..10 LOOP ... END LOOP**, és a kurzor-FOR-loop, ami automatikusan kezeli a kurzort (**FOR rec IN (SELECT id, nev FROM Diakok) LOOP ... END LOOP**). Az **SQL utasítások** közvetlenül beágyazhatók PL/SQL-be. A **SELECT INTO** egy sort kérdez le egy változóba — ha nincs találat, **NO_DATA_FOUND** kivétel; ha több

sor, `TOO_MANY_ROWS` kivétel. Az `UPDATE`, `INSERT`, `DELETE` közvetlenül használható. **DDL utasítások** (`CREATE`, `DROP`, `ALTER`) csak `EXECUTE IMMEDIATE 'SQL_string'` -gel adhatók ki. Egy **PL/SQL program** alapegysége a blokk, ami három részből áll: `DECLARE` (változók, típusok, kurzorok deklarációja), `BEGIN ... END` (a végrehajtandó utasítások, kötelező), és `EXCEPTION` (a kivételkezelés, opcionális). A blokkok egymásba ágyazhatók. **Alprogramok** közé tartoznak a tárolt eljárások és függvények. A **tárolt eljárás (PROCEDURE)** nem ad vissza értéket (vagy csak OUT paraméteren keresztül), önállóan hívható utasításként. A **tárolt függvény (FUNCTION)** mindig `RETURN` értékkel rendelkezik, kifejezésben hívható (SELECT-ben is). A két fő különbség: a procedure nem hívható közvetlenül SQL-ben, a function igen. A **csomag (PACKAGE)** logikailag összetartozó eljárások, függvények, típusok, kurzorok csoportja, két részből: a **specifikáció** mondja meg, mit lehet kívülről hívni (interface), a **body** pedig, hogyan van megvalósítva. Előnyei az encapsulation (csak a specifikációban deklarált tagok láthatók kívülről), a modularitás (összetartozó dolgok egy helyen), a teljesítmény (első hívásnál memóriába kerül), és a globális állapot támogatása. A **paraméterezési módok** három fő típusa: az **IN** (alapeset) bemeneti, csak olvasható az alprogramban; az **OUT** kimeneti, az alprogram adja vissza rajta (híváskor null-ozódik); az **IN OUT** be- és kimenet egyszerre. A hívás lehet pozicionális (`p(1, x, y)`) vagy megnevezett (`p(a => 1, b => x, c => y)`), vegyesen is. A **kivételkezelés** PL/SQL-ben az `EXCEPTION` blokkban történik. `WHEN beépített_kivétel THEN ...` formával kezelhetünk hibákat. Beépített kivételek: `NO_DATA_FOUND` (SELECT INTO nem talált sort), `TOO_MANY_ROWS` (SELECT INTO több sort kapott), `INVALID_CURSOR` (kurzor érvénytelen), `ZERO_DIVIDE` (nullával osztás), `VALUE_ERROR` (típusátalakítási hiba), `OTHERS` (minden más). Saját kivétel is definiálható: `DECLARE saját_hiba EXCEPTION;` deklarációval, majd `RAISE saját_hiba;` -val dobható és `WHEN saját_hiba THEN ...` -nal kezelhető. A `RAISE_APPLICATION_ERROR(-20001, 'üzenet')` alkalmazás-specifikus hibakód és üzenet dobására való (-20000-től -20999-ig).

12.b Számítógépi grafika

1. A **DDA szakaszrajzoló** az egyenes $y = mx + b$ alakjából lépésekkel rajzolja a pixeleket, de lebegőpontos + kerekítés miatt nem ideális.
2. A **MidPoint szakasz** csak egész számokkal: az aktuális pont után a **felezőpont** alapján dönt East vagy NorthEast pixelről.
3. A **MidPoint körrajzoló** kihasználja a **nyolcas szimmetriát** (elég a kör 1/8-át kiszámolni), és a felezőpont alapján E vagy SE pixelt választ.
4. A **Cohen-Sutherland vágó** algoritmus 4 bites kódot rendel a sík 9 részéhez; 0000 kódok → belül, AND $\neq 0$ → kívül, egyébként élvonalakkal vágunk.
5. A **Hermite-ív** két végpont és két érintővektor alapján harmadfokú polinom — négy Hermite-polinom súlyozott kombinációja.
6. A **Bézier-görbe** approximáció (csak végpont-interpoláció), **Bernstein-polinommal** előállítva — **konvex burok** tulajdonság, affin invariancia, magas fokszámánál merev.
7. A **B-Spline** lokálisan hat (új kontrollpont nem mozgatja az egész görbét), automatikus folytonossággal csatlakoznak az ívek.
8. A **homogén koordináták** egy 3D pontot $(x, y, z, 1)$ alakkal reprezentálnak — így az **eltolás is mátrixszorzással** leírható, **4×4 mátrixokkal**.
9. **Pont-transzformációk**: eltolás, forgatás, tükrözés, skálázás, nyírás; **vetítések**: párhuzamos (megtartja arányokat), centrális (perspektív, realisztikus), **axonometria**.
10. Poliédereket **Wire Frame** (csak csúcs+él) vagy **B-Rep** (geometriai+topológiai) tárol; megjelenítés: hátsó lapok eltávolítása normállal, **Z-Buffer**; árnyalás: **Flat** (lapszín), **Gouraud** (csúcs-szín interp.), **Phong** (normál interp., legszebb).

Történet

A számítógépi grafika a folytonos geometriai alakzatok képpontokkal (raszterekkel) való közelítésével kezdődik. Ezeket az algoritmusokat digitális differenciaelemző (**DDA**) algoritmusoknak hívjuk. A **DDA szakaszrajzoló** az egyenes $y = mx + b$ egyenletéből indul, és az $m = dy/dx$ meredekség segítségével határozza meg, mennyit lépjünk minden iterációban. Hátránya, hogy lebegőpontos műveleteket használ, és az (x, y) értékeket kerekíteni kell, ezért nem ad ideális képet — segíthet, ha dx és dy úgy van választva, hogy a nagyobb elmozdulás irányában legyen a növekmény 1. A **MidPoint szakasz** algoritmus csak egész számokkal dolgozik. Tegyük fel, $0 \leq m \leq 1$, balról jobbra rajzolunk; az aktuális megrajzolt $P(x_p, y_p)$ után a következő rácspont E vagy NE lehet, és a **felezőpont (M)** közelsége az elméleti egyeneshez dönt — közöttük inkrementális egész számításokkal. A **MidPoint körrajzoló** az origó középpontú r sugarú körre ($x^2 + y^2 = r^2$) épül, és kihasználja a **nyolcas szimmetria** elvét: elég a kör 1/8-át kiszámolni, a többi pont a szimmetriából adódik. Ha nem origó középpontú a kör, koordináta-transzformációt alkalmazunk. Az aktuális $P(x_p, y_p)$ után E vagy SE lehet a következő, a felezőpont alapján. A **Cohen-Sutherland vágó algoritmus** célja, hogy eldöntse, egy szakasz a képernyőablakon belül van-e. A síkot 9 részre osztjuk (a középső a képernyő), és minden részhez 4 bites kódot rendelünk (1. bit: felső vonal felett, 2. bit: alsó vonal alatt, 3. bit: jobb vonaltól jobbra, 4. bit:

bal vonaltól balra). Ha mindkét végpont kódja 0000, a szakasz belül van; ha a két kód AND-je nem 0, a szakasz ugyanazon az oldalon kívül esik; egyébként valamelyik végpontot el kell metszeni a megfelelő képernyő-éllel és újra vizsgálni. Előnye a hatékonyság és a 3D-re való általánosíthatóság (6 bites kóddal), hátránya, hogy csak téglalap-ablakra működik. A **paraméteres görbék** koordinátafüggvényekkel definiáltak: síkban $r(t) = (x(t), y(t))$, térben $+ z(t)$. Megjelenítéskor a görbét pontokon át vezetett töröttvonallal közelítjük. A **Hermite-ív** két végpontból (p_0, p_1) és két érintővektorból (t_0, t_1) építkezik harmadfokú polinommal: $s(u) = a_3u^3 + a_2u^2 + a_1u + a_0$, $u \in [0,1]$. A négy feltétel ($s(0) = p_0$, $s(1) = p_1$, $s'(0) = t_0$, $s'(1) = t_1$) megadja az együtthatókat. Az egyenletrendszer megoldása négy Hermite-polinomot ad: $H_0^3(u) = 2u^3 - 3u^2 + 1$, $H_1^3(u) = -2u^3 + 3u^2$, $H_2^3(u) = u^3 - 2u^2 + u$, $H_3^3(u) = u^3 - u^2$. A görbe: $s(u) = p_0 \cdot H_0^3 + p_1 \cdot H_1^3 + t_0 \cdot H_2^3 + t_1 \cdot H_3^3$. Hátrányai: az érintőkkel való definiálás körülményes, és bizonyos helyzetekben a görbe hurkot csinálhat vagy metszheti önmagát. A **Bézier-görbe** approximációs görbe — a görbének nem kell áthaladnia, elég megközelítenie a kontrollpontokat (csak az első és utolsó interpolálva). Bernstein-polinom segítségével állítjuk elő: $B_i^n(u) = \binom{n}{i} \cdot u^i \cdot (1-u)^{n-i}$, és $s(u) = \sum b_i \cdot B_i^n(u)$. Tulajdonságai: konvex burk-tulajdonság (a görbe a kontrollpontok konvex burkán belül marad), affin invariancia (transzformáció elvégezhető a kontrollpontokon), globális változás (egy kontrollpont mozgatása az egész görbét érinti). Magas fokszámnál merev. A **B-Spline** alacsony fokú görbéveket csatlakoztat egymáshoz, automatikus folytonossággal — egy új kontrollpont csak lokálisan hat. A bázisfüggvények 4 kontrollpontos esetben: $N_0(t) = (1-t)^3/6$, $N_1(t) = (t^3/2 - t^2 + 2/3)$, $N_2(t) = (t^3/2 + t^2/2 + t/2 + 1/6)$, $N_3(t) = t^3/6$. A **homogén koordináták** a tér pontjait olyan rendezett számhármassal reprezentálják, amik arányosság erejéig vannak meghatározva, és mind a 3 koordináta nem lehet egyszerre 0. Az áttérés Descartesről homogénra: $[x, y]$ síkban $\rightarrow [x, y, 1]$ homogén alak. Térben $(x, y, z, 1)$ alakú négyes, és a transzformációkat **4×4-es mátrixokkal** írjuk le. Visszaalakítás: $[x_1/x_3, x_2/x_3, 1]$. Cél, hogy az eltolás is mátrixszorzással leírható legyen. A **pont-transzformációk** geometriai leképezések. Az eltolás: $p' = p + v$. A forgatás α szöggel egy tengely körül; tetszőleges tengely körüli forgatás felbontható a 3 koordináta-tengely körüli forgatás kompozíciójára. A tükrözés a koordináta-síkokra; a koordinátasík nem szereplő tengelyéhez tartozó koordinátát negáljuk. Az origó középpontú nyújtás-zsugorítás minden tengely irányban azonos arányban; a skálázás eltérő arányban. A nyírás a tér pontjait egy adott síkkal párhuzamosan csúsztatja. A **window-viewport transzformáció** a képsíkon lévő ablakot átviszi a képernyőn megjelenítendő ablakba — eltolás origóba, skálázás, eltolás végleges pozícióba. A tér leképezése síkra **vetítéssel** történik ($R^3 \rightarrow R^2$). A **párhuzamos vetítés** a térbeli p pontot egy adott v irányvektor mentén tolja a képsíkig — megtartja az egyeneseket, párhuzamosságot és arányokat, de a távolságot és szöget nem. A **centrális (perspektív) vetítés** a centrumot (kamera) és a vetítés irányát igényli — a párhuzamos szelők tétele alapján a távolabbi objektumok kisebbnek látszanak. Az **axonometria** adott térbeli koordináta-rendszerhez (origó, e_1, e_2, e_3 egységvektorok) és ezek képeihez (origó', e_1', e_2', e_3') rendel pontokat: a p pont koordinátáit megszorozzuk a megfelelő egységvektor-képekkel és összegezzük. Színes, árnyalt **poliéder-megjelenítéshez** a felületeket polygon hálóval közelítjük (általában háromszögekkel, mert 4 pont nem feltétlenül egy síkban van). A **Wire Frame modell** csak a csúcokat és éleket tárolja — a kép sokszor értelmezhetetlen, és a nem látható részek nem távolíthatók el. A **B-Rep (Boundary Representation)** továbblép: tárolja a geometriai (csúcskoordináták, élek és lapok egyenletei) és topológiai (lapok, élek, csúcspontok kapcsolatai) információkat. Az **hátsó lapok eltávolítása** során a lap normálvektorát (vektoriális szorzat), majd a normál és a kamerába mutató vektor szögét (skaláris szorzat) vizsgáljuk — ha 90° -nál

nagyobb, a lap háttal van, eltávolítható. A háromszögeket súlypont z koordinátája szerint rendezzük (módosított gyorsrendezéssel), és hátulról kezdjük megjeleníteni, hogy az elülsők kitakarják a hátsókat. A **Z-Buffer algoritmus** pixelenként vizsgálja, melyik objektum látszik. A mélység-puffert maximális mélységre, a pixeleket háttérszínre inicializáljuk; bejárjuk az objektumokat, és minden pixelen, ha az aktuális pont z koordinátája közelebb van a tárolt értékénél, frissítjük a szín- és mélység-értéket. Az árnyalási technikák a természetes megvilágítás közelítésére valók. A **flat (konstans) árnyalás** minden lap egy pontjában számít egy színértéket, és azt használja az egész lapra; ideális képet ritka (párhuzamos fény, végtelen kamera, valódi poliéder), és görbült felület-közelítéseknél ugrásszerű színváltások jelentkeznek a szomszédos lapok közt. A **Gouraud árnyalás** minden csúcsban kiszámítja a színt a normálisokból, és a lap belső pontjait kettős lineáris interpolációval színezi; hátrányai a sima körvonal hiánya és a nagy tükröződéseknel jelentkező ugrás. A **Phong árnyalás** a csúcsokban tárolt **normálisokat interpolálja** (nem a színeket), és minden belső pontban kiszámítja a normál alapján a színt — több számítást igényel, de szebb eredményt ad.

13.a Webprogramozás II — PHP

1. A **statikus weboldal** mindig ugyanazt küldi; a **dinamikus** futás közben generálja a tartalmat felhasználó/adatbázis alapján (PHP, Python, Node.js).
2. A **PHP** szerver-oldali, dinamikusan típusos, interpretált nyelv — C-szerű szintaxis, HTML-be ágyazható (WordPress, Drupal).
3. A futáshoz **webszerver** (Apache/Nginx) és **PHP interpreter** kell — a klasszikus **LAMP stack**: Linux + Apache + MySQL + PHP.
4. **Típusrendszer**: skalár (int, float, string, bool), összetett (array, object, callable), speciális (null, resource) — a típus futás közben változhat.
5. A változó `$` prefixxel, a **konstans** `define()` -al vagy `const` -tal; **superglobálisok**: `$_GET` , `$_POST` , `$_SESSION` , `$_COOKIE` , `$_FILES` , `$_SERVER` .
6. **HTML-PHP kapcsolat**: `<?php ?>` blokkokkal — a PHP feldolgozza a kódot, a többi változatlanul küldi a kliensnek.
7. **Modulszerkezet**: `include` / `require` (utóbbi fatális hiba hiányzó fájlnál), `*_once` (csak egyszer), **PSR-4 autoloading** modern projekteknel.
8. **Adatcsere**: GET URL-ben (látható, cache-elhető), POST body-ban (rejtett), **AJAX** aszinkron a böngészőből, **JSON** az API-knál.
9. **Biztonsági kérdések**: SQL Injection ellen **prepared statement**; XSS ellen `htmlspecialchars` ; CSRF ellen token + SameSite cookie; jelszó `password_hash` / `password_verify` .
10. A **süti (cookie)** kliensen tárolt 4 KB-ig terjedő adat (HttpOnly, Secure, SameSite flag-ekkel); a **session** szerveroldali állapot, csak a session ID megy cookie-ban.

Történet

A **dinamikus weboldalak** abban különböznek a statikusaktól, hogy a szerver futás közben generálja a választ (felhasználó, paraméter, adatbázis alapján), míg a statikus mindig ugyanazt küldi (egyszerű HTML fájlt). A statikus oldal gyors és egyszerű, de nem személyre szabható; a dinamikus interaktív (bejelentkezés, kosár, fórum), de komplexebb és lassabb. A **PHP** (PHP: Hypertext Preprocessor) szerver-oldali, dinamikusan típusos, interpretált programozási nyelv, amit eredetileg webfejlesztésre terveztek. Jellemzői a C-szerű szintaxis, a HTML-be közvetlenül ágyazhatóság, nagy közösség, sok keretrendszer (Laravel, Symfony), és beépített webhez kapcsolódó függvények (cookie, session, DB, fájl). Az ismert WordPress, Drupal, Magento mind PHP-ban íródott. A futtatás feltételei: a szerveren legyen **PHP interpreter** és **webszerver** (Apache, Nginx) konfigurálva PHP-feldolgozásra (gyakran FastCGI/PHP-FPM-en keresztül); az adatbázis (általában MySQL/MariaDB) opcionális. A klasszikus **LAMP stack** négy darab szabad szoftverből áll: **L**inux + **A**pache + **M**ySQL + **P**HP. A kliens-szerver infrastruktúrában a kliens oldal a böngészőben fut (HTML struktúra, CSS kinézet, JavaScript interaktivitás), a szerver oldalon pedig a webszerver, a PHP interpreter, az adatbázis, és gyakran cache (Redis, Memcached). Egy életciklus: a böngésző HTTP kérést küld a szervernek (pl. `GET /index.php`); a webszerver fogadja, és PHP fájl esetén az interpreternek továbbítja; az interpreter értelmezi és

futtatja a kódot; a generált HTML válasz visszamegy a kliensnek; a böngésző megjeleníti. A PHP **típusrendszere** négy skalár típust (int, float, string, bool), összetett típusokat (array — szám- vagy string-kulcsos asszociatív tömb, ami egyben lista is; object; callable), és speciális típusokat (null, resource) ismer. A nyelv **dinamikusan típusos**: egy változó típusa futás közben változhat (pl. `$x = 5;` `$x = "öt";` legitim). A változók `$` jellel kezdődnek (`$nev = "Anna";`). A **konstansokat** `define()` függvénnyel vagy `const` kulcsszóval definiáljuk (`define('PI', 3.14);` vagy `const VERSION = '1.0';`); a konstanson nem `$` jel áll. Vannak **predefinált változók (superglobálisok)**: `$_GET` (URL paraméterek), `$_POST` (form mezők), `$_REQUEST` (GET + POST + COOKIE), `$_SESSION` (szerveroldali állapot), `$_COOKIE` (kliens oldali sütik), `$_FILES` (feltöltött fájlok), `$_SERVER` (szerver és kliens info). A **HTML és PHP kapcsolat** úgy működik, hogy a PHP közvetlenül HTML-be ágyazható `<?php ... ?>` blokkokkal; a feldolgozáskor a `<?php ?>` részeket az interpreter értelmezi, a többit változatlanul küldi a kliensnek. **Modulszerkezet** és kód-újrahasznosítás eszközei: az `include 'fajl.php';` beemeli egy másik PHP fájl tartalmát (hiányzó fájlnál figyelmeztetés); a `require 'fajl.php';` ugyanaz, de hiányzó fájl esetén **fatális hiba** (állj); az `include_once` és `require_once` csak egyszer emelnek be akkor sem, ha többször hívánk (osztálydefinícióknál fontos). A strukturálásnál érdemes a konfigurációt, az adatbázis-kapcsolatot, és a header/footer komponenseket külön fájlba tenni; modern megközelítés a **PSR-4 autoloading**, ami névtér alapján automatikusan tölt be osztályfájlokat. Az **adatcsere** a kliens és a szerver között többféleképpen történhet. A HTML formok `method="GET"` vagy `method="POST"` attribútummal küldhetnek adatot. A **GET** az URL-ben szállít paramétereket (látható, könyvjelzőzhető, cache-elhető — adatlekérdezéshez); a `$_GET` változón keresztül érjük el. A **POST** a HTTP body-ban szállít (nem látszik az URL-ben, nem cache-elt — adat-módosításhoz); a `$_POST` változón keresztül. Az **AJAX** lehetővé teszi, hogy a JavaScript aszinkron HTTP-kéréseket küldjön a szervernek anélkül, hogy az egész oldalt újra kellene tölteni. A **JSON** a modern szerver-kliens kommunikáció formátuma (REST API-knál): `header('Content-Type: application/json'); echo json_encode([ ... ]);`. A **biztonsági kérdések** kritikusak. Az **SQL Injection** akkor lehetséges, ha a felhasználói input közvetlenül SQL-be kerül; védekezés: **prepared statement** használata, ahol a query és a paraméter szétválasztva megy a DB felé. Az **XSS (Cross-Site Scripting)** akkor lép fel, ha a kliens input visszaíródik a HTML-be JavaScript-ként; védekezés: `htmlspecialchars($input, ENT_QUOTES, 'UTF-8')` a kimeneten. A **CSRF (Cross-Site Request Forgery)** védelméhez a form-okban CSRF token kell, amit a szerver ellenőriz, és cookie-knál `SameSite` attribútum használata javasolt. A jelszavakat **soha nem szabad tisztán tárolni**, hanem a `password_hash($jelszo, PASSWORD_BCRYPT)` (vagy argon2) függvénnyel hash-elve, és `password_verify($jelszo, $hash)` -fel ellenőrizve. Minden adatot ellenőrizz a szerveren (a kliens-validációban ne bíz). HTTPS használata kötelező. File upload-nál ellenőrizd a típust, és ne mentsd futtatható mappába. A **munkafolyamatok** és felhasználói állapot kezelésére a **cookie (süti)** és a **session** szolgál. A **cookie** kis adatdarabot tárol a kliens böngészőjében, és minden kéréssel visszaküldi a szervernek (`setcookie("user", "anna", time() + 3600);` 1 órás cookie). Mérete ~4 KB-ig, manipulálható a kliens által. Fontos flag-ek: **HttpOnly** (JavaScriptből nem érhető el, XSS védelem), **Secure** (csak HTTPS-en megy át), **SameSite** (CSRF védelem). A **session** szerveroldali állapot-tárolás: a kliens csak egy session ID-t tárol cookie-ban (vagy URL-ben), az adat a szerveren van. `session_start();` után `$_SESSION['user_id'] = 42;` írható és olvasható. Nem manipulálható a klientsől (csak a session ID lopható), és bejelentkezett állapot, kosár tartalom, stb. tárolható benne.

13.b Programozási technológiák — tervezési minták

1. A **tervezési minta** újrahasznosítható megoldás egy gyakori problémára — közös szókincs, bevált megoldás, karbantarthatóság; a **Gang of Four** 23 mintát rendszerezett.
2. **Három fő kategória: létrehozási** (objektum-példányosítás), **szerkezeti** (összeszerelés), **viselkedési** (kommunikáció).
3. A **Stratégia** minta interface-t és cserélhető implementációkat ad, a kontextus **kompozícióval** tartja a stratégiát — futás közben váltható.
4. A **Sablon metódus** minta absztrakt osztály template metódusával adja az algoritmus vázát; a változó lépéseket absztrakt metódusként hagyja — **öröklődéssel** működik.
5. A Stratégia és a Sablon metódus különbsége: az előbbi **kompozíció + futás-közbeni csere**, az utóbbi **öröklődés + fordításkori rögzítés**.
6. A **Megfigyelő (Observer)** egy alany állapotváltozásairól értesít minden megfigyelőt anélkül, hogy ismerné őket — fajtái **push** (adatot is ad) és **pull** (csak értesít).
7. A **SOLID** alapelvek: Single Responsibility, **Open/Closed**, Liskov Substitution, Interface Segregation, Dependency Inversion.
8. A **nyitva-zárt alapelv** szerint a kód bővítésre nyitott, módosításra zárt — új viselkedés új osztály, a meglévőket nem nyúljuk; **interface + polimorfizmus** a kulcs.
9. **GOF1**: programozz interfészre, ne implementációra; **GOF2**: részesítsd előnyben a kompozíciót az öröklődéssel szemben.
10. A **TDD három lépése** Red-Green-Refactor: bukó teszt, minimális kód a zöldhöz, majd szépítés — kiegészíti a code review, CI/CD, refactoring, naming conventions.

Történet

A **tervezési minták** újrahasznosítható megoldások gyakori tervezési problémákra. Nem konkrét kódot adnak, hanem **sémákat** — sok helyzetben adaptálhatók. Előnyeik: közös szókincs (a fejlesztők gyorsabban értik egymást), bevált megoldások (sokan tesztelték), karbantarthatóság, bővíthetőség. A **Gang of Four (GoF)** könyv 1994-ben 23 mintát rendszerezett három fő kategóriába. A **létrehozási (creational) minták** az objektum-példányosítást teszik rugalmasabbá: **Singleton** (Egyke — egyetlen példányt biztosít), **Factory Method** (Gyár-metódus — a leszármazottak döntenek el a konkrét típust), **Abstract Factory** (Absztrakt gyár — kapcsolódó objektum-családokat), **Builder** (Építő — komplex objektumot lépésről lépésre), **Prototype** (Prototípus — klónozással). A **szerkezeti (structural) minták** az objektumok és osztályok összeszerelését strukturálják: **Adapter** (két inkompatibilis interfész illesztése), **Decorator** (futás közben bővítés), **Proxy** (helyettesít, vezérli a hozzáférést), **Bridge**, **Composite** (fa-hierarchia), **Flyweight** (sok hasonló objektum hatékony tárolása), **Facade** (egységes felület bonyolult alrendszerhez). A **viselkedési (behavioral) minták** az objektumok kommunikációját szervezik: **Strategy** (algoritmus-családot cserélhetővé tesz), **Template Method** (algoritmus váza, lépéseket leszármazottak adják), **Observer** (esemény-értesítés), **Iterator** (kollekció bejárása), **Command** (kérést objektummá csomagol — undo, queue), **State** (állapot-függő viselkedés), **Chain of Responsibility** (kérés végigfut egy láncon), **Mediator**, **Visitor**. A **Stratégia és a Sablon metódus**

minta hasonlít, de van egy alapvető különbség. A **Stratégia** egy interface-t és több implementációt ad: a kliens **kompozícióval** tartja a stratégia-példányt, és **futás közben válthatja** (`SetStrategy()`). A **Sablon metódus** egy absztrakt osztály template metódusát adja, ami megírja az algoritmus vázát, és a változó lépéseket absztrakt metódusként hagyja; a leszármazott implementálja a részleteket — vagyis **öröklődéssel** dolgozik, és a választás **fordításkor rögzített**. A különbségek összefoglalva: a Stratégia mechanizmusa kompozíció, futás közben váltható, az algoritmus teljes a stratégiában; a Sablon metódus mechanizmusa öröklődés, fordításkor rögzített, és csak vázát adja az algoritmusnak, a részleteket a leszármazottakra bízva. A **Megfigyelő (Observer) minta** egy alany (Subject) objektum állapotváltozásairól értesít minden megfigyelőt (Observer), anélkül, hogy ezek konkrét típusát ismernék. Két fajta: a **push** változatban az alany az adatot is átadja a Notify-ban, a **pull** változatban csak értesít, az observer maga kérdezi le, ha kell. Felhasználása GUI eseménykezelés, MVC, pub-sub minta, .NET event-jei. **Egyéb gyakori minták**: az **Egyke (Singleton)** csak egy példányt enged egy típusból, privát konstruktorral és statikus Instance property-vel (logger, konfiguráció, DB kapcsolat-pool); a **Díszítő (Decorator)** egy objektumot egy másikkal vesz körbe, ami kibővíti a viselkedését (pl. stream tömörített/titkosított rétegek); az **Adapter** két inkompatibilis interface-t illeszt össze (régi API új API-hoz); az **Iterátor** a kollekció bejárás mechanizmusát különválasztja (C#-ban beépített IEnumerable és foreach); a **Parancs (Command)** kérést objektummá csomagol, ami undo, queue, log lehet rá. Az **objektum-orientált tervezési alapelvek** szerepe is fontos. A **SOLID** öt alapelvet rejt: **Single Responsibility** (egy osztálynak egy felelőssége legyen), **Open/Closed** (bővítésre nyitott, módosításra zárt), **Liskov Substitution** (ős helyett mindig használható a leszármazott), **Interface Segregation** (kis, fókuszált interface-ek), **Dependency Inversion** (absztrakcióktól függünk, ne konkrétól). A **nyitva-zárt alapelv** mondja, hogy a kód **bővítésre nyitott, módosításra zárt** legyen — új funkciókat tudunk hozzátenni, anélkül, hogy a meglévő, már működő és tesztelt kódot változtatnánk. Megvalósítása: interface-ek és absztrakt osztályok + polimorfizmus — egy új viselkedés egy új implementáló osztály, a meglévőeket nem kell módosítani. A **GOF1 alapelv**: programozz interfészre, ne implementációra — a változókat és paramétereket interface típusra írjuk, ne konkrét osztályra (pl. `IList<int>` lista helyett `List<int>`). Ez lehetővé teszi az implementáció későbbi cseréjét anélkül, hogy a klienskódot módosítani kellene, és csökkenti az osztályok közötti csatolást. A **GOF2 alapelv**: részesítsd előnyben a kompozíciót az öröklődéssel szemben. Az öröklődés erős, fordításidőben rögzített kapcsolat — futás közben nem változtatható; a kompozíció rugalmasabb, futás közben cserélhető. Egyéb alapelvek: **DRY** (Don't Repeat Yourself — ne ismételd kódot), **KISS** (Keep It Simple, Stupid — a legegyszerűbb működő megoldás a legjobb), **YAGNI** (You Aren't Gonna Need It — csak azt írd meg, amire most kell), és **Demeter törvénye** (csak a közvetlen szomszédoknak küldj üzenetet, ne beszélj idegenekkel). A **jól bevált módszerek** szerepe a szoftverfejlesztésben kritikus. A **TDD (Test-Driven Development)** három lépésből áll: **Red** (írd egy tesztet, ami megbukik), **Green** (írd annyi kódot, hogy a teszt átmenjen — minimális megoldás), **Refactor** (szépítsd a kódot, miközben a tesztek átmennek). Előnyei: automatikusan lefedett a kód tesztekkel, a kód tesztelhető szerkezetű lesz, korai visszajelzés a hibákról, és refactor-bátorság. Egyéb jól bevált módszerek: **code review** (minden kódot átnéz egy másik fejlesztő), **continuous integration (CI)** (minden commitnál automatikus build + teszt), **continuous deployment (CD)** (zöld buildek élesre kerülnek), **páros programozás, refactoring** (kód javítása viselkedés-megőrzéssel), **naming conventions** (beszédesebb, konzisztens nevek), **verziókezelés** (Git, branch stratégiák), és **dokumentáció** (kódba közvetlenül, vagy README/JSDoc).

14.a Programozási technológiák — OOP elvek + minták

1. Az **osztály felület** (mit kínál kívülről) és **megvalósítás** (hogyan oldja meg) két részből áll — a felület stabil, a megvalósítás cserélhető.
2. Az **objektum három jellemzője**: felület (publikus metódusai), **belső állapot** (privát mezők értéke), **viselkedés** (hogyan reagál a hívásokra).
3. Az **OOP négy alapelve** a fenti fogalmakkal: egységbe záras (adat + művelet együtt), adatrejtés (állapot csak felületen át), öröklődés (felület + megvalósítás), polimorfizmus (azonos felület, eltérő viselkedés).
4. A **GOF1** alapelv interface-re programozást ír elő — a változó típusa legyen a legabsztraktabb, így a konkrét osztály cserélhető.
5. A **GOF2** alapelv a **kompozíciót** részesíti előnyben az öröklődéssel szemben — futás közben cserélhető, lazább kapcsolat, mint az "is-a".
6. A **Stratégia minta** közös interface + kontextust ad, ami kompozícióval tartja a stratégia-példányt; `SetStrategy()` metódussal cserélhető — **nyitva-zárt** elv teljesül.
7. **Viselkedési minták általában**: Strategy, Template Method, Observer, Iterator, Command, State, Chain of Responsibility, Mediator, Visitor.
8. Az **Egyke (Singleton)** garantálja az egyetlen példányt: **privát konstruktor**, **static Instance property**, lazy init lock-kal (vagy `Lazy<T>`) — logger, konfiguráció, connection pool.
9. **Létrehozási minták általában**: Singleton, Factory Method, Abstract Factory, Builder, Prototype.
10. A **Díszítő (Decorator)** ugyanazt az interface-t implementálja, mint a díszítendő, körbeveszi, és funkciót ad hozzá (Compress, Encrypt) — **szerkezeti minták**: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.

Történet

A programozási technológiák magasabb absztrakciós szinten elemzik az objektum-orientált tervezést. Az **osztály** két részből áll: a **felület (interface, kontraktus)** mondja meg, mit kínál kívülről — milyen publikus szolgáltatások állnak rendelkezésre, publikus metódusok és property-k szignatúrái formájában; a többi osztály kívülről ezt látja. A **megvalósítás (implementation)** a felület mögötti tényleges kód: a privát mezők, segédmetódusok — a felhasználó számára rejtett, és bármikor változhat anélkül, hogy a klienseket befolyásolná. Az **objektum**, ami az osztály konkrét példánya, három aspektussal jellemezhető. A **felülete** a publikus metódusai, amikkel kívülről kommunikálunk vele — megegyezik az osztály felületével. A **belső állapota** a privát mezők aktuális értékei; két azonos osztályú objektum különböző állapotban lehet. A **viselkedése** az, hogyan reagál a metódushívásokra; a viselkedést a megvalósítás határozza meg, de függhet a belső állapottól (pl. üres verem `pop()`-jára kivetelt dob). Az OOP négy alapelvét ezekkel a fogalmakkal újfogalmazhatjuk. Az **egységbe záras** (encapsulation): az adat (belső állapot) és a rajta értelmezett műveletek (viselkedés) egy egységben (osztály) vannak; a felület közvetít kifelé. Az **adatrejtés** (information hiding): a belső állapot közvetlenül nem elérhető kívülről, csak a publikus felületen át — így biztosítható, hogy az állapot mindig konzisztens, és lehetővé teszi a megvalósítás cseréjét. Az **öröklődés**: új osztály öröklí egy

meglévő felületét és megvalósítását, és kibővítheti vagy specializálhatja. A **polimorfizmus**: ugyanaz a felület-művelet különböző viselkedést mutathat az osztály konkrét típusától függően; megvalósítása virtual + override, késői kötés (dinamikus diszpéc). Ez lehetővé teszi, hogy a kliens csak a felülettel dolgozzon. A **GOF1 alapelv** szerint programozz interfészre, ne implementációra — a változók, paraméterek típusát mindig a legabsztraktabb interface-en vagy absztrakt osztályon adjuk meg, amit elég; a konkrét típus csak a példányosításnál érdekes. Például `IList<int> lista = new List<int>()` jobb, mint `List<int> lista = ...`. Előnyei: a konkrét implementáció később cserélhető anélkül, hogy a klienseket módosítani kellene; a tesztelés könnyebb (mock-olható az interface); csökken az osztályok közti csatolás (coupling). A **GOF2 alapelv** szerint részesítsd előnyben a kompozíciót az öröklődéssel szemben. Az öröklődés (is-a kapcsolat) erős, fordításidőben rögzített, az ős változása mindenki leszármazottját érinti, és nehezen változtatható a viselkedés futás közben — az egyszeres öröklődés korlátozó. A kompozíció (has-a kapcsolat): egy objektum birtokol egy másikat, és a birtokolt objektum futás közben cserélhető — rugalmasabb, lazább kapcsolat. A Stratégia minta klasszikus alkalmazása a kompozíciónak. A **Stratégia minta részletes bemutatása**: a probléma az, hogy sok hasonló algoritmus létezik (pl. rendezés, fizetés módja), és futás közben szeretnénk váltani közöttük, anélkül, hogy a kliens kódját módosítanánk. A megoldás: definiálj egy közös interface-t az algoritmusoknak (pl. `IRendezo` `Rendezz()` metódussal), implementáld őket külön osztályokban (`Buborek : IRendezo`, `Quicksort : IRendezo`), és a kliens (`Kontextus`) kompozíción keresztül tartson egy interface-példányt. Egy `SetStrategy(IRendezo s)` metódussal futás közben változtatható. Előnye: új algoritmus = új osztály, a Kontextus változatlan — a nyitva-zárt alapelv teljesül. A **viselkedési minták** általában az objektumok közötti kommunikációt és felelősség-elosztást strukturálják. Példák: Strategy (cserélhető algoritmusok), Template Method (algoritmus váza öröklődéssel), Observer (esemény-értesítés), Iterator (kollekció bejárása), Command (kérés objektumba csomagolva), State (állapot-függő viselkedés), Chain of Responsibility (kérés végigfut egy láncon, valaki kezeli), Mediator (közvetítő), Visitor (művelet objektumhierarchián). Az **Egyke (Singleton) minta részletes bemutatása**: szeretnénk garantálni, hogy egy osztályból csak egyetlen példány legyen az alkalmazás futása során, és bárhonnan elérhető legyen. A megoldás kulcselemei: **privát konstruktor** (kívülről nem példányosítható), **statikus Instance property** (a globális elérési pont), **lazy init lock-kal** (szálbiztos egyszeri inicializálás, double-checked locking), és **sealed** (ne lehessen megkerülni leszármazottal). Modern C#-ban a `Lazy<T>` típussal egyszerűbb: `private static readonly Lazy<Logger> _instance = new Lazy<Logger>(() => new Logger());`. Használata: logger, konfiguráció, DB connection pool, cache. Mikor kerüljük: ha tesztelhetőség fontos (mocking nehéz), ha valójában csak globális változót akarunk (anti-pattern) — modernben sokszor dependency injection-nel váltjuk ki. A **létrehozási minták** általában az objektum-példányosítást teszik rugalmasabbá: Singleton (egyetlen példány), Factory Method (leszármazottak döntenek el a konkrét típust), Abstract Factory (kapcsolódó objektum-családok), Builder (komplex objektumot lépésről lépésre), Prototype (meglévő példányt klónozik). A **Díszítő (Decorator) minta részletes bemutatása**: a probléma az, hogy egy objektumhoz futás közben szeretnénk plusz felelősséget rendelni, anélkül, hogy az osztály-hierarchiát módosítanánk vagy hatalmas leszármazott-fát építenénk. A megoldás: a díszítő osztály ugyanazt az interface-t implementálja, mint a díszítendő objektum, és körbeveszi azt — előtte/utána tehet pluszt. Példa: `IStream` interface-szel `FileStream : IStream`; egy `StreamDecorator : IStream` absztrakt alaposztály egy belső `IStream`-mel; `CompressDecorator` és `EncryptDecorator` leszármazottak, amik az írást előbb tömörítik/titkosítják, majd átengedik a

belső stream-nek. Használata: `IStream s = new EncryptDecorator(new CompressDecorator(new FileStream()))`; . Előnyei: futás közben kombinálható a viselkedés; nyitva-zárt (új díszítő = új osztály); kombinációk exponenciális robbanása elkerülhető (öröklődéssel exp. lenne). A .NET `Stream` osztály-családja klasszikusan így működik (GZipStream, CryptoStream stb.). A **szerkezeti minták** általában az objektumok és osztályok összeszerelését strukturálják: Adapter (két inkompatibilis interface illesztése), Bridge (absztrakció és implementáció szétválasztása), Composite (fa-szerű hierarchia), Decorator (futás közbeni bővítés), Facade (egységes egyszerű felület bonyolult alrendszerhez), Flyweight (sok hasonló objektum hatékony tárolása megosztott állapottal), Proxy (helyettesít, kontrollálja a hozzáférést — cache, lazy load, biztonság).

14.b Adatbázis II — kurzor + trigger + kollekció

1. A **kurzor** a memóriában lévő környezeti terület azonosítója, ami az SQL eredményét tárolja és mutatóval halad a sorokon.
2. **Implicit kurzor** automatikusan jön létre DML-utasításokhoz és 1 sort visszaadó SELECT-hez; attribútumai `SQL%FOUND`, `SQL%NOTFOUND`, `SQL%ROWCOUNT`, `SQL%ISOPEN` (mindig FALSE).
3. **Explicit kurzor 4 lépése:** `DECLARE` (`CURSOR név IS ...`), `OPEN` (lefuttatja a lekérdezést), `FETCH` (egy sort betölt), `CLOSE` (felszabadít).
4. Az **egyszerűsített FOR-LOOP** automatikusan kezeli az open-fetch-close-t: `FOR rec IN (SELECT ... ) LOOP ... END LOOP`.
5. **Explicit kurzor-attribútumok:** `%FOUND` (sikeres FETCH után TRUE), `%NOTFOUND`, `%ISOPEN`, `%ROWCOUNT` (eddig betöltött sorok); nincs megnyitva → `INVALID_CURSOR` kivétel.
6. A **kurzorváltozó (REF CURSOR)** dinamikus — futás közben bármely kompatibilis lekérdezéshez kapcsolható; **erős** (RETURN típussal) vagy **gyenge** (típus nélkül).
7. **Tranzakciókezelés:** `COMMIT` véglegesít, `ROLLBACK` visszagörget, `SAVEPOINT` köztes mentés; DDL implicit COMMIT-tal jár.
8. A **trigger** automatikusan végrehajtódó kód egy eseményre (tábla módosulás, rendszer); típusai **sor-** vs **utasításszint**, **BEFORE/AFTER/INSTEAD OF**, rendszer-trigger.
9. **Trigger-felhasználás:** naplózás, integritás, származtatott érték frissítése, üzleti szabály — sor-szintű triggerben `:NEW` és `:OLD` érhető el.
10. **Kollekciók 3 típusa:** **asszociatív tömb** (hash, csak PL/SQL, int/string index), **beágyazott tábla** (DB oszlop is lehet, lyukak, törölhető elem), **VARRAY** (fix max méret, folytonos, nem törölhető elem).

Történet

Az adatbázis-programozás egyik kulcsfogalma a **kurzor**, ami egy SQL-utasítás eredményének iterálását teszi lehetővé. Egy SQL-utasítás feldolgozásához az Oracle a memóriában egy **környezeti területet** használ, ami információkat tartalmaz az utasítás által feldolgozott sorokról: lekérdezésnél az aktív halmazt (visszaadott sorokat) és egy mutatót az utasítás belső reprezentációjára. A kurzor megnevezi ezt a környezeti területet, és lehetővé teszi az aktív halmaz sorainak egyenkénti elérését — mint a telefonkönyvben keresés az ujjunkkal, ahol az ujj a kurzor. Kétféle kurzor van: implicit és explicit. Az **implicit kurzor** a PL/SQL által automatikusan felépített kurzor minden DML-utasításhoz (INSERT, UPDATE, DELETE) és minden olyan SELECT-hez, ami pontosan egy sort ad vissza (SELECT INTO). Attribútumai a legutoljára végrehajtott utasításra vonatkoznak: `SQL%FOUND` TRUE, ha a DML legalább egy sort érintett, vagy SELECT INTO visszaadott sort; `SQL%NOTFOUND` ennek ellentéte; `SQL%ROWCOUNT` az érintett sorok száma; `SQL%ISOPEN` mindig FALSE, mert az Oracle automatikusan lezárja az implicit kurzort az utasítás után. Az **explicit kurzor** a több sort visszaadó lekérdezések eredményének kezelésére szolgál; a programozó kezeli négy lépésben. Az első lépés a **deklaráció**: `CURSOR cur_nev IS SELECT ... ;`. A második a **megnyitás**: `OPEN cur_nev;` — lefuttatja a lekérdezést, meghatározódik az aktív halmaz, és a kurzormutató az első rekordra áll; egy

újra megnyitott kurzor a `CURSOR_ALREADY_OPEN` kivételt váltja ki. A harmadik a **sorok betöltése**: `FETCH cur_nev INTO v1, v2, ...;` — a mutatóval címzett sort betölti a változóba, és a mutatót a következő sorra állítja; nem létező sor betöltése nem vált ki kivételt. A negyedik a **lezárás**: `CLOSE cur_nev;` — érvényteleníti a kurzor és az aktív halmaz közötti kapcsolatot. Egy egyszerűsített formára is van lehetőség: az **egyszerűsített FOR-LOOP** automatikusan kezeli az open-fetch-close-t: `FOR rec IN (SELECT id, nev FROM Diakok) LOOP DBMS_OUTPUT.PUT_LINE(rec.nev); END LOOP;`. Az explicit kurzor **attribútumai** csak procedurális utasításokban használhatók, SQL-ben nem. **%FOUND**: az első sor betöltése előtt NULL, sikeres FETCH után TRUE, egyébként FALSE. **%NOTFOUND** ennek ellentéte. **%ISOPEN** TRUE, ha a kurzor meg van nyitva. **%ROWCOUNT** az első sor betöltése előtt 0, ezután mindig 1-gyel nő, megadja a betöltött sorok darabszámát. Ha az explicit kurzor vagy a kurzorváltozó nincs megnyitva, akkor a %ISOPEN kivételével a másik három az `INVALID_CURSOR` kivételt váltja ki. A **kurzorváltozó (REF CURSOR)** explicit kurzorhoz hasonlóan aktív halmaz valamely sorának feldolgozására alkalmas, de **dinamikus**: futási időben bármely típuskompatibilis kérdés hozzákapcsolható. Lényegében egy referencia típusú változó, amely a hivatkozott sor címét tartalmazza. Deklarálás: `TYPE nev IS REF CURSOR [RETURN típus];` majd `cur nev;`. Ha van RETURN, akkor **erős** kurzorreferencia (a fordító ellenőrzi a kompatibilitást); ha nincs, akkor **gyenge** (bármely kérdés kapcsolható). A **tranzakciókezelés** a 7.b-ben tárgyalt ACID tulajdonságokon alapul. PL/SQL-ben a **COMMIT** véglegesíti, a **ROLLBACK** visszagörgeti a változásokat; a **SAVEPOINT** köztes mentési pontot ad, ahova rollback-elhetünk. Implicit COMMIT történik DDL utasításnál (CREATE, ALTER, DROP) és felhasználói kilépéskor; implicit ROLLBACK ha az alkalmazás rendellenesen leáll. A **triggererek** olyan tevékenységek, amik automatikusan végbemennek, ha egy tábla vagy nézet módosul, vagy egyéb felhasználói/rendszeresemény bekövetkezik. A trigger adatbázis-objektum, és PL/SQL-ben (vagy Java/C-ben) írható meg. Jellemzői: a felhasználó számára átlátszó (nem hívja, csak észreveszi a hatást); a tárolt eljárások egy speciális fajtája, amelynek lefutása nem hagyható ki; táblához vagy adatbázis-eseményhez kötődnek; mindig abban a táblában érvényes és hajtódik végre, amelyhez kötöttük, és INSERT/UPDATE/DELETE után. Egy táblának több triggerere lehet, és összetettebb eseményekre is alkalmazhatók. Felhasználása: naplózás (audit — ki, mit, mikor módosított); integritási megszorítások összetett esetekben; származtatott értékek automatikus frissítése; üzleti szabályok kikényszerítése (pl. nem lehet hétvégén rendelni). Korlátai: sok memóriát és időt igényelhet; túlhasználata teljesítményproblémát okoz; eldugott logika nehezíti a hibakeresést. **Trigger-típusok**: a **sorszintű** trigger akkor fut le, amikor a tábla adatai módosulnak — DELETE esetén minden törölt sor aktiválja. Az **utasításszintű** egyszer fut le, az érintett sorok számától függetlenül. **BEFORE/AFTER**: a BEFORE a kapcsolt utasítás előtt fut, az AFTER utána; mindkettő lehet sor- vagy utasításszintű, és csak táblához kapcsolható (nézethez nem). **INSTEAD OF**: a kapcsolt utasítás helyett fut le, csak sorszintű, és csak nézeteken definiálható. **Rendszer-triggererek**: adatbázis-eseményekre (rendszeresemények, indítás, leállítás, serverhiba, felhasználói események, bejelentkezés, kijelentkezés, DDL utasítások). A szintaxis: `CREATE [OR REPLACE] TRIGGER trignevev BEFORE/AFTER/INSTEAD OF esemény ON tabla [FOR EACH ROW] [WHEN (feltétel)] BEGIN ... END;`. Sor-szintű triggerekben elérhetők a `:NEW` és `:OLD` referenciák — a sor új és régi értékei. A **kollekciók** azonos típusú adatelemek rendezett együttese, minden elemnek egyedi indexével. Paraméterként átadhatók, így a táblák oszlopai mozgathatók az alkalmazás és az adatbázis között. Három típus van. Az **asszociatív tömb** hash-tábla típusú, indexei (kulcsai) tetszőleges egészek vagy stringek lehetnek, és az indexnek nincsenek határai. Deklarálható PL/SQL blokkban, alprogramban, csomagban, **de csak PL/SQL**

programokban használható (adatbázis oszlop típusa nem lehet). A **beágyazott tábla** elemei lehetnek objektumtípus példányai, és ők lehetnek objektumtípus attribútumai. Az index alsó határa 1. Deklarálható PL/SQL-ben vagy adatbázis-objektumként (**CREATE TYPE**). Adatbázistábla oszlopának típusa is lehet (pl. ügyfel tábla könyvek oszlopa). Az elemek szétszórtan helyezkednek el, és **lehetnek lyukak** közöttük. Új elem tetszőleges indexű helyre vihető be, és bármely elem törölhető. A **dinamikus tömb (VARRAY)** szintén lehet objektumtípus elemekkel és adatbázis-objektum. Az index alsó határa 1. Deklarációjához **meg kell adni a maximális méretet**, ez lesz az index felső határa. Bővíthető új elemekkel, de a meglévők csak cserélhetők, **nem törölhetők**. Az elemek a kisebb indextől indulva **folytonosan helyezkednek el**.